Laporan Tugas Besar 1
IF3070 Dasar Intelegensi Artifisial 2026/2027



Kelompok 1

AGNDSN

Nizham Rafa Lazuardi	**18224049**
Adrian Akhdan Assyauqi	**18224055**
Muhammad Azikra Wira Pratama	**18224089**

PROGRAM STUDI SISTEM DAN TEKNOLOGI INFORMASI

SEKOLAH TEKNIK ELEKTRO DAN INFORMATIKA

INSTITUT TEKNOLOGI BANDUNG

2026

Daftar Isi

Daftar Isi **2**
Deskripsi Persoalan **4**
Daftar Variabel Permasalahan 4
Ketentuan Representasi 4
Objective Function & Constraints 5
Pembahasan **6**
Implementasi Algoritma 6
Struktur Repository 6
Steepest Ascent Hill Climbing 8
Sideways Move Hill Climbing 9
Random Restart Hill Climbing 11
Stochastic Hill Climbing 13
Simulated Annealing 14
Genetic Algorithm 16
Penjelasan models.py 20
Penjelasan problems.py 21
Penjelasan neighbors.py 23
Insialisasi State 27
Hasil Eksperimen (Nilai Objective: Shipping Fee) 31
Steepest Ascent Hill Climbing 31
Sideways Move Hill Climbing 32
Random Restart Hill Climbing 34
Stochastic Hill Climbing 36
Simulated Annealing 37
Genetic Algorithm 42
Hasil Eksperimen (Nilai Objective: ETA) 60
Steepest Ascent Hill Climbing 60
Sideways Move Hill Climbing 62
Random Restart Hill Climbing 64
Stochastic Hill Climbing 67
Simulated Annealing 69
Genetic Algorithm 72
Analisis Hasil 90
Pertanyaan Analisis 91
Kesimpulan dan Saran **96**
Kesimpulan 96
Saran 96
Referensi **97**

Pembagian Tugas..... 98
SURAT PERNYATAAN PENGGUNAAN AI..... 99

Deskripsi Persoalan

Tugas ini bertujuan untuk membuat konfigurasi penempatan kendaraan paling optimal pada sebuah geladak kapal kargo. Setiap kendaraan dan geladak direpresentasikan sebagai grid dua dimensi berbentuk persegi atau persegi panjang. Solusi dapat memilih kendaraan mana yang diangkut dan menentukan posisi serta orientasinya di dalam geladak, dengan tetap memenuhi seluruh batasan.

Daftar Variabel Permasalahan

Entitas	Atribut	Contoh	Keterangan
Vehicle	id	“Fuad”	*Identifier*
	Dimensions	{w:3, l:4}	Ukuran sebuah kendaraan, terdiri atas *width* dan *length* (satuan grid)
	Orientation	Horizontal	*Horizontal* atau *Vertical*
	ShippingFee	60	Tarif/keuntungan pengiriman kendaraan (satuan juta rupiah)
	Weight	5	Berat kendaraan (satuan unit)
	ETA	3	Estimated Time of Arrival, perkiraan waktu kedatangan kendaraan (dalam sisa hari)
Ship	Dimensions	{w:10, l:20}	Ukuran geladak pada kapal, terdiri atas *width* dan *length* (satuan grid)
	MaxCapacity	500	Kapasitas angkut maksimum suatu kapal (satuan unit)

Ketentuan Representasi

Berikut prosedur bagaimana dalam merepresentasikan permasalahan ini:

- Kendaraan dan geladak kapal direpresentasikan sebagai grid dua dimensi berbentuk persegi atau persegi panjang.
- Kendaraan dapat berbeda ukuran satu sama lain dan tidak semua harus dimasukkan ke dalam peti kemas pada kapal sehingga suatu kendaraan dapat berada di luar **(seluruh bagian kendaraan di luar kapal)**
- Koordinat berupa bilangan bulat. Cara representasi koordinat dibebaskan.
- Struktur data yang digunakan untuk representasi dibebaskan. Misalnya menggunakan *key-value* seperti JSON.

- Inisialisasi (***initial state***) random.
- Langkah neighbor (***move***) yang diperbolehkan tiap iterasi:
    1. Menukar dua kendaraan, baik di dalam maupun di luar kapal
    2. Memindahkan sebuah kendaraan ke suatu koordinat yang berbeda
    3. Mengubah orientasi kendaraan
- Representasi solusi (***final state***) adalah pemetaan semua paket yang ada dan juga pemetaan semua koordinat dalam peti kemas pada kapal. Secara detail yang harus dapat ditunjukkan oleh state dan visualisasi adalah:
    1. Bagaimana keadaan semua kendaraan yang di dalam maupun yang di luar
    2. Bagaimana keadaan geladak pada kapal
    3. *Value* pada *state* tersebut

Objective Function & Constraints

Agar permasalahan ini tidak menjadi kompleks, berikut Objective Function & Constraints yang ditetapkan

- **Objective function**
  Permasalahan ini bertujuan untuk memaksimalkan *ShippingFee* pada kapal.
- **Kendaraan tidak boleh saling bertumpuk (*no overlapping*)**
  Tidak boleh ada satu sel grid geladak yang ditempati oleh lebih dari satu kendaraan.
- **Kendaraan harus berada di dalam batas geladak.**

  Seluruh sel grid yang ditempati suatu kendaraan harus berada di dalam dimensi geladak kapal.
- **Kapal tidak boleh mengangkut melebihi kapasitasnya.**

Pembahasan

Implementasi Algoritma

Permasalahan diselesaikan dengan berbagai algoritma local search, mencakup 4 variasi Hill Climbing, Simulated Annealing, dan Genetic Algorithm. Pembahasan mencakup alur kerja, tahapan implementasi, serta mekanisme pencarian solusi pada masing-masing algoritma berdasarkan kode program yang telah dikembangkan.

Struktur Repository

```
├── src/                              # Source code utama aplikasi
│   ├── algorithms/                   # Implementasi algoritma pencarian
│   │   ├── brute_force.py
│   │   ├── genetic.py
│   │   ├── random_restart.py
│   │   ├── sideways.py
│   │   ├── simulated_annealing.py
│   │   ├── steepest.py
│   │   └── stochastic.py
│   ├── static/                       # Aset antarmuka web
│   │   ├── css/style.css
│   │   └── js/                       # Logika antarmuka dan grafik
│   ├── templates/index.html          # Template halaman web
│   ├── app.py                        # Aplikasi Flask
│   ├── models.py                     # Model data
│   ├── neighbors.py                  # Operasi untuk mencari solusi tetangga
│   ├── problem.py                    # Input, validasi, dan evaluasi solusi
│   └── runner.py                     # Menjalankan eksperimen
├── data/example.json                 # Contoh data masukan
├── docs/                             # Dokumentasi tambahan
├── output/                           # Hasil eksperimen dalam format JSON
├── requirements.txt                  # Dependensi Python
└── README.md                         # Dokumentasi utama proyek
```

File	Deskripsi Singkat
/src/app.py	Menyediakan halaman Flask dan API untuk menjalankan algoritma serta membuat penempatan kendaraan secara acak.
/src/models.py	Mendefinisikan struktur data kapal, kendaraan, penempatan, dan state solusi.
/src/neighbors.py	Menghasilkan solusi tetangga melalui operasi pindah, tukar, dan rotasi kendaraan.
src/problem.py	Memuat data persoalan, memvalidasi

	penempatan, menghitung nilai objektif, dan membuat state awal.
src/runner.py	Memilih dan menjalankan algoritma, lalu menyusun metrik dan hasil eksperimen.
/src/algorithms/brute_force.py	Mencari solusi terbaik dengan mengevaluasi kemungkinan penempatan secara menyeluruh.
/src/algorithms/genetic.py	Mengoptimalkan penempatan kendaraan menggunakan seleksi dan persilangan dalam algoritma genetika.
/src/algorithms/random_restart.py	Menjalankan steepest ascent berulang kali dari state awal yang berbeda.
/src/algorithms/sideways.py	Menjalankan hill climbing yang mengizinkan perpindahan ke solusi dengan nilai sama.
/src/algorithms/simulated_annealing.py	Mencari solusi dengan simulated annealing dan peluang menerima solusi yang lebih buruk.
/src/algorithms/steepest.py	Memilih tetangga terbaik pada setiap langkah steepest ascent hill climbing.
/src/algorithms/stochastic.py	Menjalankan hill climbing dengan memilih kandidat tetangga secara acak.
src/static/css/style.css	Mengatur tampilan formulir, dek kapal, panel hasil, dan grafik.
/src/static/js/app.js	Menghubungkan formulir, API, visualisasi dek, dan tampilan hasil eksperimen.
/src/static/js/charts.js	Menggambar grafik nilai objektif dan proses simulated annealing.
/src/static/js/deck.js	Menampilkan petak dek, posisi kendaraan, dan kendaraan yang berada di luar kapal.
src/static/js/form.js	Mengelola input kapal dan kendaraan, state awal, serta impor, ekspor, dan pengacakan data.
/src/templates/index.html	Menyusun halaman untuk konfigurasi algoritma, input persoalan, visualisasi dek, dan hasil eksperimen.

Steepest Ascent Hill Climbing

Steepest Ascent Hill Climbing adalah algoritma yang bertujuan mencari solusi dengan memilih seluruh neighbor yang skor-nya lebih besar dari current. Hal ini terus dilakukan berulang kembali hingga tidak ada skor yang lebih tinggi lagi. Algoritma ini akan berhenti ketika tidak ada neighbor yang lebih tinggi dari current terbaru. Berikut adalah alur implementasi algoritma Steepest Ascent Hill Climbing adalah sebagai berikut:

1. Melakukan inisialisasi current state berdasarkan initial state, kemudian menghitung skor awal dan memasukkannya ke dalam array objective history serta menginisialisasi jumlah iterasi menjadi 0.
2. Melakukan looping untuk seluruh neighbor berdasarkan current state. Setiap neighbor akan dihitung skornya dan dibandingkan dengan skor terbaik yang tercatat saat itu. Jika skor neighbor lebih tinggi, maka best neighbor akan diperbarui. Jika terdapat neighbor lain dengan skor yang sama dengan skor terbaik dan lebih tinggi dari current state, maka neighbor tersebut juga dimasukkan ke dalam daftar kandidat.
3. Hasil dari looping akan diperiksa. Jika tidak ditemukan neighbor dengan skor yang lebih tinggi dari current state, maka algoritma berhenti. Namun, jika ditemukan, salah satu neighbor terbaik akan dipilih secara acak untuk menggantikan current state.
4. Setelah current state diperbarui, jumlah iterasi bertambah satu dan skor terbaik dimasukkan ke dalam array objective history. Proses pencarian kemudian diulangi hingga tidak ditemukan neighbor dengan skor yang lebih tinggi.

```
import random

from ..models import State, Ship, Vehicle
from ..problem import shipping_fee_fitness
from ..neighbors import iter_neighbors

# Inisialisasi Function
def steepest_ascent(initial_state: State, ship: Ship, vehicles: dict[str,
Vehicle], evaluate=shipping_fee_fitness, rng: random.Random | None = None) ->
dict:
    # Jika rng kosong, generate rng baru
    if rng is None:
        rng = random.Random()

    # Inisialisasi variabel
    current = initial_state
    objective_history = [evaluate(current, vehicles)]
    iterations = 0
```

```
    # Mulai looping untuk mencari state yang lebih baik
    while True:
        # Menghitung current score
        current_score = evaluate(current, vehicles)

        # Inisialisasi tempat untuk kandidat neighbors yang akan dipilih
        best_neighbors = []
        best_score = current_score

        # Cari seluruh neighbor dan hitung score nya
        for neighbor in iter_neighbors(current, ship, vehicles):
            neighbor_score = evaluate(neighbor, vehicles)

            # Catat neighbor yang lebih baik dari current state
            if neighbor_score > best_score:
                best_neighbors = [neighbor]
                best_score = neighbor_score
            elif neighbor_score == best_score and neighbor_score > current_score:
                best_neighbors.append(neighbor)

        # Stop jika tidak ada neighbor yang lebih baik dari current state
        if not best_neighbors:
            break

        # Pilih random untuk menjadi state terbaik dari kandidat neighbors yang
sudah dicatat
        current = rng.choice(best_neighbors)
        iterations += 1
        # Catat
        objective_history.append(best_score)

    return {
        "state": current,
        "iterations": iterations,
        "iteration_history": list(range(iterations + 1)),
        "objective_history": objective_history,
    }
```

Sideways Move Hill Climbing

Sideways Move Hill Climbing adalah algoritma modifikasi dari steepest ascent hill climbing, namun dengan perbedaan bahwa neighbor terbaik dapat berpindah walaupun bernilai sama dan tidak ke state yang sama. Hal ini berupaya untuk menghindari local optimum jika terus-menerus mencari value yang lebih tinggi. Algoritma ini akan berhenti pada iterasi maksimal yang telah ditentukan. Berikut adalah alur implementasi algoritma Sideways Move Hill Climbing adalah sebagai berikut:

1. Melakukan looping untuk neighbor berdasarkan current state. Hasil for loop tersebut menghasilkan best neighbor dengan syarat bahwa valuenya tertinggi atau sama dengan dari value best neighbor yang terekam saat itu serta state-nya belum pernah dikunjungi.

2. Hasil dari looping akan mengubah state current dengan state best neighbor lalu iterasi bertambah, melakukan perekaman skor terbaik dimasukkan ke array, dan state yang pernah dikunjungi direkam.
3. Lalu melakukan pemeriksaan skor, jika skor terbaru lebih besar maka jumlah gerakan dimulai dari 0, namun jika sebaliknya maka jumlah gerakan terus bertambah satu.
4. Memeriksa jumlah gerakan, jika sudah melebihi maksimal maka algoritma berhenti.

```
from ..models import State, Ship, Vehicle
from ..problem import shipping_fee_fitness
from ..neighbors import iter_neighbors


def sideways(initial_state: State, ship: Ship, vehicles: dict[str, Vehicle],
evaluate=shipping_fee_fitness, max_sideways_moves: int = 10000) -> dict:
    current = initial_state
    objective_history = [evaluate(current, vehicles)]
    iterations = 0
    sideways_moves = 0

    # Value untuk menyimpan state yang sudah pernah dikunjungi
    visited_states = set()
    visited_states.add(current.key())

    while True:
        current_score = evaluate(current, vehicles)

        best_neighbor = None
        best_score = current_score

        # For loop untuk menjalankan seluruh kemungkinan neighbor serta menentukan
neighbor terbaik
        for neighbor in iter_neighbors(current, ship, vehicles):
            neighbor_score = evaluate(neighbor, vehicles)

            # Jika skor neighbor dan current sama serta state sudah pernah
dikunjungi keluar for loop
            if neighbor_score == current_score and neighbor.key() in
visited_states:
                continue

            # Membandingkan neighbor dan best skor saat ini, jika neighbor skor
lebih besar akan mengisi value best skor dan best neighbor
            if (neighbor_score >= best_score):
                best_neighbor = neighbor
                best_score = neighbor_score

        if best_neighbor is None:
            break

        # Hasil best neighbor dari for loop mengisi current value
        current = best_neighbor
        iterations += 1
        objective_history.append(best_score)
        visited_states.add(current.key())
```

```
        # Jika tidak ada perkembangan skor, jumlah gerak bertambah. Jika
sebaliknya jumlah gerak dimulai dari 0
        if best_score == current_score:
            sideways_moves += 1
        else:
            sideways_moves = 0

        # While loop berakhir ketika sudah gerakan sudah mencapai max
        if sideways_moves >= max_sideways_moves:
            break

    return {
        "state": current,
        "iterations": iterations,
        "sideways_moves": sideways_moves,
        "iteration_history": list(range(iterations + 1)),
        "objective_history": objective_history,
    }
```

Random Restart Hill Climbing

Random Restart Hill Climbing adalah pengembangan dari algoritma Steepest Ascent Hill Climbing yang bertujuan untuk meningkatkan peluang menemukan solusi global optimum dengan melakukan pencarian berulang dari berbagai initial state yang di generate random oleh sistem. Alur implementasi algoritma Random Restart Hill Climbing adalah sebagai berikut:

1. Menjalankan pencarian awal menggunakan algoritma Steepest Ascent Hill Climbing dengan initial_state yang diberikan, kemudian menyimpan hasilnya dalam variabel first_search.
2. Menginisialisasi variabel untuk menyimpan state terbaik sementara, nilai fungsi objektif terbaik, riwayat nilai objektif, riwayat iterasi, dan jumlah iterasi pada setiap restart.
3. Melakukan proses random restart sebanyak max_restarts. Pada setiap pengulangan, algoritma generate initial state secara random menggunakan fungsi random_state, kemudian menjalankan kembali algoritma Steepest Ascent Hill Climbing.
4. Membandingkan nilai fungsi objektif yang diperoleh selama pencarian dengan best_score. Jika ditemukan nilai yang lebih tinggi, algoritma memperbarui nilai terbaik dan menyimpan state terbaik yang ditemukan.
5. Memperbarui data historis pencarian, meliputi jumlah iterasi dan perkembangan nilai fungsi objektif terbaik selama seluruh proses restart.
6. Mengembalikan hasil pencarian berupa state terbaik, total iterasi, jumlah restart yang dilakukan, jumlah iterasi pada setiap restart, serta riwayat iterasi dan nilai fungsi objektif untuk keperluan analisis.

```
import random

from ..models import State, Ship, Vehicle
```

```
from ..problem import random_state, shipping_fee_fitness
from .steepest import steepest_ascent

# Inisialisasi Function
def random_restart(initial_state: State, ship: Ship, vehicles: dict[str, Vehicle],
rng: random.Random, evaluate=shipping_fee_fitness, max_restarts: int = 100) ->
dict:
    # Inisialisasi Variabel
    first_search = steepest_ascent(initial_state, ship, vehicles, evaluate, rng)
    best_state = first_search["state"]
    best_score = evaluate(best_state, vehicles)
    objective_history = first_search["objective_history"].copy()
    iteration_history = first_search["iteration_history"].copy()
    total_iterations = first_search["iterations"]
    iterations_per_restart = []

    # Loop untuk mengulangi algoritma steepest sebanyak max_restarts dengan state
awal random
    for _ in range(max_restarts):
        # Generate random state lalu mencari state lebih baik menggunakan
algoritma steepest
        current = random_state(ship, vehicles, rng)
        search = steepest_ascent(current, ship, vehicles, evaluate, rng)
        iterations_per_restart.append(search["iterations"])

        # Membandingkan state awal dengan best state sementara
        start_score = search["objective_history"][0]
        if start_score > best_score:
            best_state = current
            best_score = start_score
        objective_history.append(best_score)
        iteration_history.append(total_iterations)

        # Mencari score terbaik
        for score in search["objective_history"][1:]:
            total_iterations += 1
            if score > best_score:
                best_score = score
            objective_history.append(best_score)
            iteration_history.append(total_iterations)

        # Mengganti best_state jika ditemukan state yang lebih baik
        if search["objective_history"][-1] > evaluate(best_state, vehicles):
            best_state = search["state"]

    # Mengembalikan state, total iterasi yang dilakukan, restart yang dilakukan,
banyak iterasi per restart, data historis iterasi, dan data historis nilai fitness
    return {
        "state": best_state,
        "iterations": total_iterations,
        "restart_count": len(iterations_per_restart),
        "iterations_per_restart": iterations_per_restart,
        "iteration_history": iteration_history,
        "objective_history": objective_history,
    }
```

Stochastic Hill Climbing

Stochastic Hill Climbing adalah algoritma modifikasi dari steepest ascent hill climbing, namun dengan perbedaan bahwa neighbor dibuat secara acak kemudian dibandingkan skor saat ini. Jika skor neighbor lebih besar maka value akan diganti dengan neighbor tersebut. Algoritma ini akan berhenti hingga iterasi yang telah ditentukan berkahir. Berikut adalah alur implementasi algoritma Stochastic Hill Climbing adalah sebagai berikut:

1. Melakukan inisialisasi current state berdasarkan initial state, kemudian menghitung skor awal dan memasukkannya ke dalam array objective history serta menginisialisasi jumlah iterasi menjadi 0.
2. Melakukan looping berdasarkan jumlah iterasi maksimal yang telah ditentukan. Pada setiap iterasi, algoritma akan menghasilkan satu random neighbor berdasarkan current state.
3. Melakukan pemeriksaan terhadap random neighbor yang dihasilkan. Jika neighbor tersedia dan memiliki skor lebih tinggi dibandingkan current state, maka current state akan diperbarui menjadi neighbor tersebut. Jika tidak memenuhi kondisi tersebut, maka current state tidak berubah.
4. Setelah pemeriksaan selesai, jumlah iterasi bertambah satu dan skor current state terbaru dimasukkan ke dalam array objective history.
5. Proses tersebut diulangi hingga mencapai jumlah iterasi maksimal yang telah ditentukan. Setelah itu, algoritma berhenti dan mengembalikan current state, jumlah iterasi, serta riwayat iterasi dan skor yang telah direkam.

```
import random
from ..models import State, Ship, Vehicle
from ..problem import shipping_fee_fitness
from ..neighbors import random_neighbor


def stochastic(initial_state: State, ship: Ship, vehicles: dict[str, Vehicle],
evaluate=shipping_fee_fitness, rng: random.Random | None = None, max_iterations:
int = 100) -> dict:
    if rng is None:
        rng = random.Random()

    current = initial_state
    objective_history = [evaluate(current, vehicles)]
    iterations = 0

    # Looping sesuai iterasi yang ditentukan
    for _ in range(max_iterations):
        # Membuat value random neighbor
        neighbor = random_neighbor(current, ship, vehicles, rng)

        # Hasil skor prioritas neighbor dibandingkan dengan skor prioritas saat
ini, current value diganti dengan neighbor
        if neighbor is not None and evaluate(neighbor, vehicles) >
evaluate(current, vehicles):
```

```
            current = neighbor

        iterations += 1
        # Skor priotas terbaru direkam dan dimasukkan ke array
        objective_history.append(evaluate(current, vehicles))


    return {
        "state": current,
        "iterations": iterations,
        "iteration_history": list(range(iterations + 1)),
        "objective_history": objective_history,
    }
```

Simulated Annealing

Simulated Annealing adalah algoritma yang menggunakan pendekatan probabilistik untuk meningkatkan peluang menemukan solusi global optimum. Algoritma ini memiliki kemiripan dengan Stochastic Hill Climbing, tetapi memungkinkan perpindahan ke state dengan nilai fungsi objektif yang lebih rendah berdasarkan probabilitas tertentu. Alur implementasi algoritma simulated annealing adalah sebagai berikut:

1. Menginisialisasi variabel untuk menyimpan current state, temperatur awal, riwayat nilai fungsi objektif, riwayat probabilitas penerimaan, serta data iterasi yang mencapai optimum lokal.
2. Memulai proses looping dengan batas maksimum max_iterations. Pada setiap iterasi, algoritma memeriksa apakah temperatur masih berada di atas batas minimum yang ditentukan.
3. Generate satu random neighbor dari current state menggunakan fungsi random_neighbor sebagai kandidat solusi berikutnya.
4. Menghitung perubahan nilai fungsi objektif (deltaE) berdasarkan selisih nilai fungsi objektif neighbor terhadap current state.
5. Menentukan penerimaan neighbor. Jika deltaE > 0, neighbor langsung diterima sebagai current state. Jika sebaliknya, neighbor diterima berdasarkan probabilitas = e^(deltaE/T) , yang dibandingkan dengan bilangan acak pada rentang 0 hingga 1.
6. Memeriksa kondisi optimum lokal dengan membandingkan nilai fungsi objektif current state terhadap seluruh neighbor yang tersedia. Jika tidak terdapat neighbor dengan nilai fungsi objektif lebih tinggi, iterasi tersebut dicatat dalam stuck_iterations.
7. Memperbarui data historis dan temperatur dengan mencatat nilai fungsi objektif current state, kemudian menurunkan temperatur menggunakan cooling rate dengan persamaan temperatur baru = temperatur lama * cooling rate yang bernilai 0.995 secara default.

8. Menghentikan proses pencarian apabila jumlah iterasi mencapai max_iterations, temperatur mencapai batas minimum yang di set (default = 0.01), atau tidak terdapat neighbor yang dapat dibangkitkan.
9. Mengembalikan hasil pencarian berupa state akhir, total iterasi, riwayat nilai fungsi objektif, riwayat probabilitas penerimaan, serta jumlah dan frekuensi iterasi ketika algoritma berada pada local optimum.

```
import math
import random

from ..models import State, Ship, Vehicle
from ..neighbors import iter_neighbors, random_neighbor
from ..problem import shipping_fee_fitness

# Inisialisasi Function
def simulated_annealing(initial_state: State, ship: Ship, vehicles: dict[str,
Vehicle], rng: random.Random, evaluate=shipping_fee_fitness, initial_temperature:
float = 90.0, cooling_rate: float = 0.995, max_iterations: int = 2000) -> dict:
    # Inisialisasi Variabel
    current = initial_state
    temperature = initial_temperature
    objective_history = [evaluate(current, vehicles)]
    eET_history = []
    stuck_iterations = []
    local_optimum_cache = {}

    # Memulai looping dengan dibatasi maks iterasi
    for iteration in range(1, max_iterations + 1):
        # Memberhentikan loop jika temperatur berada dibawah batas minimum
(default: 0.01)
        if temperature <= 0.01:
            break

        # Generate random neighbor
        neighbor = random_neighbor(current, ship, vehicles, rng)

        # Berhenti jika neighbor yang digenerate kosong
        if neighbor is None:
            break

        # Menghitung deltaE
        delta = evaluate(neighbor, vehicles) - evaluate(current, vehicles)

        # Jika deltaE > 0, ganti current state dengan neighbor sekarang
        if delta > 0:
            current = neighbor
            eET_history.append(None)
        # Jika tidak, hitung eET untuk menjadi probabilitas neighbor terpilih
        else:
            eET = math.exp(delta / temperature)
            eET_history.append(eET)
            if rng.random() < eET:
                current = neighbor

        # Hitung current_score
```

```
        current_score = evaluate(current, vehicles)

        # Mengecek apakah local optimum
        state_key = current.key()
        if state_key not in local_optimum_cache:
            local_optimum_cache[state_key] = all(
                evaluate(candidate, vehicles) <= current_score
                for candidate in iter_neighbors(current, ship, vehicles)
            )
        if local_optimum_cache[state_key]:
            stuck_iterations.append(iteration)

        # Mencatat nilai fitness
        objective_history.append(current_score)

        # Mengurangi temperature
        temperature *= cooling_rate

    # Menghitung total iterasi
    iterations = len(objective_history) - 1

    # Mengembalikan state yang terpilih, banyak iterasi, data historis iterasi,
data historis nilai fitness, data historis eET. total stuck
    return {
        "state": current,
        "iterations": iterations,
        "iteration_history": list(range(iterations + 1)),
        "objective_history": objective_history,
        "eET_history": eET_history,
        "stuck_iterations": stuck_iterations,
        "stuck_count": len(stuck_iterations),
        "stuck_frequency": len(stuck_iterations) / iterations if iterations else
0,
    }
```

Genetic Algorithm

Genetic Algorithm adalah algoritma pencarian yang terinspirasi dari proses evolusi dan seleksi alam. Algoritma ini menggunakan sekumpulan solusi yang disebut populasi, kemudian menghasilkan solusi baru melalui proses *selection*, *crossover*, dan mutasi. Pada tugas besar ini, *Genetic Algorithm* digunakan untuk mencari susunan kendaraan pada *deck* kapal yang menghasilkan keuntungan pengiriman sebesar mungkin.

Implementasi algoritma terdapat pada src/algorithms/genetic.py, dengan fungsi utama genetic_algorithm(). Tahapan pencariannya adalah sebagai berikut.

1. **Pembentukan populasi awal**
   Populasi awal dibuat menggunakan fungsi random_state() yang menghasilkan sejumlah susunan kendaraan secara acak. Setiap individu direpresentasikan sebagai objek State yang menyimpan posisi dan orientasi kendaraan. Jika terdapat *initial state* dari pengguna, susunan tersebut dimasukkan sebagai salah satu individu dalam populasi.

2. **Evaluasi *fitness* dan pemilihan induk**
   Setiap individu dievaluasi menggunakan fungsi *fitness*, yaitu shipping_fee_fitness() untuk menghitung total keuntungan pengiriman. Selanjutnya, fungsi select_parent() memilih dua induk menggunakan metode *fitness-proportionate selection*, sehingga individu dengan nilai *fitness* lebih tinggi memiliki peluang lebih besar untuk terpilih.
3. ***Crossover***
   Kedua induk digabungkan menggunakan metode *one-point crossover*, yaitu dengan mengambil sebagian susunan kendaraan dari induk pertama dan sisanya dari induk kedua. Hasil penggabungan kemudian diperbaiki menggunakan fungsi repair() agar memenuhi batasan kapal, seperti kapasitas berat, ukuran *deck*, dan kendaraan yang tidak boleh bertumpuk.
4. **Mutasi**
   Setelah *crossover*, individu baru memiliki peluang mutasi sebesar 10%. Proses ini menggunakan fungsi random_neighbor() untuk mengubah susunan kendaraan secara acak, seperti memindahkan kendaraan, menukar posisi dua kendaraan, atau mengubah orientasinya. Mutasi membantu menambah variasi solusi yang dicari.
5. **Pembentukan generasi baru**
   Proses *selection*, *crossover*, dan mutasi diulang hingga jumlah individu pada populasi baru mencapai population_size. Populasi baru kemudian dievaluasi untuk menghitung nilai *fitness* maksimum dan rata-rata. Individu dengan nilai *fitness* tertinggi pada setiap generasi disimpan dalam final_best_state.
6. **Kondisi berhenti dan hasil pencarian**
   Algoritma berhenti setelah mencapai jumlah iterasi yang ditentukan. Fungsi genetic_algorithm() mengembalikan individu terbaik dari populasi generasi terakhir, riwayat nilai *fitness* maksimum dan rata-rata, serta informasi jumlah populasi, iterasi, dan durasi pencarian. Data tersebut digunakan untuk visualisasi dan analisis hasil eksperimen.

```
import random
from time import perf_counter

from ..models import Placement, Ship, State, Vehicle
from ..neighbors import random_neighbor
from ..problem import footprint, is_valid, random_state, shipping_fee_fitness

# Salindia tidak menyebutkan apakah function harus return best historical score
atau current score
# Jadi ambil referensi dari https://www.geeksforgeeks.org/dsa/genetic-algorithms/
untuk return best historical score.

# Memilih parent berdasarkan nilai fitness
def select_parent(population: list[State], scores: list[float], rng:
random.Random) -> State:
    if sum(scores) == 0:
        return rng.choice(population)

    return rng.choices(population, weights=scores, k=1)[0]
```

```
# Memperbaiki state hasil crossover agar valid
def repair(child: State, ship: Ship, vehicles: dict[str, Vehicle], rng:
random.Random, evaluate=shipping_fee_fitness) -> State:
    repaired = State({vehicle_id: None for vehicle_id in vehicles})
    priorities = {}

    for vehicle_id, vehicle in vehicles.items():
        priority_state = State({key: None for key in vehicles})
        priority_state.placements[vehicle_id] = child.placements[vehicle_id]
        priorities[vehicle_id] = evaluate(priority_state, vehicles)

    ordered_ids = sorted(vehicles, key=priorities.get, reverse=True)

    for vehicle_id in ordered_ids:
        placement = child.placements[vehicle_id]
        vehicle = vehicles[vehicle_id]
        alternatives = []

        for rotated in (False, True):
            width, length = footprint(vehicle, rotated)
            for y in range(ship.length - length + 1):
                for x in range(ship.width - width + 1):
                    candidate = Placement(x, y, rotated)
                    if candidate != placement:
                        alternatives.append(candidate)

        rng.shuffle(alternatives)
        candidates = []
        if placement is not None:
            candidates.append(placement)
        candidates.extend(alternatives)

        for candidate in candidates:
            repaired.placements[vehicle_id] = candidate
            if is_valid(repaired, ship, vehicles):
                break
            repaired.placements[vehicle_id] = None

    return repaired


# Menggabungkan dua parent menggunakan one-point crossover
def crossover(parent1: State, parent2: State, ship: Ship, vehicles: dict[str,
Vehicle], rng: random.Random, evaluate=shipping_fee_fitness) -> State:
    vehicle_ids = list(vehicles)
    if len(vehicle_ids) < 2:
        return State(parent1.placements.copy())

    cut = rng.randrange(1, len(vehicle_ids))
    placements = {}

    for index, vehicle_id in enumerate(vehicle_ids):
        if index < cut:
            placements[vehicle_id] = parent1.placements[vehicle_id]
        else:
```

```
            placements[vehicle_id] = parent2.placements[vehicle_id]

    return repair(State(placements), ship, vehicles, rng, evaluate)


# Fungsi utama Genetic Algorithm
def genetic_algorithm(ship: Ship, vehicles: dict[str, Vehicle], population_size:
int, iterations: int, rng: random.Random, evaluate=shipping_fee_fitness,
initial_state: State | None = None) -> dict:
    started = perf_counter()
    mutation_probability = 0.1
    population = []
    if initial_state is not None:
        population.append(State(initial_state.placements.copy()))

    while len(population) < population_size:
        population.append(random_state(ship, vehicles, rng))

    scores = [evaluate(state, vehicles) for state in population]
    initial_best_index = scores.index(max(scores))
    initial_best_state = State(population[initial_best_index].placements.copy())
    final_best_state = State(initial_best_state.placements.copy())
    max_fitness_history = [max(scores)]
    avg_fitness_history = [sum(scores) / len(scores)]

    for _ in range(iterations):
        new_population = []

        while len(new_population) < population_size:
            parent1 = select_parent(population, scores, rng)
            parent2 = select_parent(population, scores, rng)
            child = crossover(parent1, parent2, ship, vehicles, rng, evaluate)

            if rng.random() < mutation_probability:
                mutated = random_neighbor(child, ship, vehicles, rng)
                if mutated is not None:
                    child = mutated

            new_population.append(child)

        population = new_population
        scores = [evaluate(state, vehicles) for state in population]
        generation_best_index = scores.index(max(scores))
        generation_best_fitness = scores[generation_best_index]
        final_best_state =
State(population[generation_best_index].placements.copy())
        max_fitness_history.append(generation_best_fitness)
        avg_fitness_history.append(sum(scores) / len(scores))

    runtime_ms = (perf_counter() - started) * 1000

    return {
        "initial_state": initial_best_state,
        "best_state": final_best_state,
        "max_fitness_history": max_fitness_history,
        "avg_fitness_history": avg_fitness_history,
        "population_size": population_size,
```

```
        "iterations": iterations,
        "runtime_ms": runtime_ms,
    }
```

Penjelasan models.py

models.py digunakan untuk mendefinisikan struktur data yang dipakai dalam permasalahan penempatan kendaraan pada kapal. Terdapat empat kelas utama, yaitu Vehicle untuk menyimpan informasi kendaraan, Ship untuk menyimpan informasi kapal, Placement untuk menyimpan posisi dan orientasi kendaraan, serta State untuk menyimpan susunan seluruh kendaraan. Setiap kelas menggunakan @dataclass agar pembuatan dan penyimpanan objek menjadi lebih sederhana.

```
from dataclasses import dataclass

# Menyimpan data kendaraan beserta ukuran, berat, biaya pengiriman, dan estimasi
waktu kedatangan
@dataclass
class Vehicle:
    id: str
    width: int
    length: int
    weight: float
    shipping_fee: float
    eta: int

# Menyimpan ukuran deck kapal dan kapasitas maksimum muatan kapal
@dataclass
class Ship:
    width: int
    length: int
    max_capacity: float

# Menyimpan posisi kendaraan pada deck kapal dan apakah kendaraan dirotasi
@dataclass(frozen=True)
class Placement:
    x: int
    y: int
    rotated: bool

# Menyimpan konfigurasi penempatan seluruh kendaraan, None jika kendaraan tidak
ditempatkan
@dataclass
class State:
    placements: dict[str, Placement | None]

    def key(self):
        return tuple(sorted(self.placements.items()))
```

Penjelasan problems.py

problem.py berisi fungsi yang digunakan untuk mengolah data permasalahan, mulai dari membaca input kapal dan kendaraan, memeriksa apakah suatu state valid, hingga menghitung nilai fungsi objektif. Terdapat dua fungsi evaluasi, yaitu shipping_fee_fitness() untuk menghitung total keuntungan pengiriman dan eta_fitness() untuk menghitung nilai prioritas kendaraan berdasarkan ETA. File ini juga menyediakan fungsi untuk mengubah state menjadi grid dan membuat initial state secara acak.

```
import json
import math
import random
from pathlib import Path

from .models import Placement, Ship, State, Vehicle

# Mengambil ukuran kendaraan sesuai orientasi
def footprint(vehicle: Vehicle, rotated: bool) -> tuple[int, int]:
    if rotated:
        return vehicle.length, vehicle.width
    return vehicle.width, vehicle.length

# Membaca dan memvalidasi data input kapal dan kendaraan
def problem_from_data(data) -> tuple[Ship, dict[str, Vehicle]]:
    ship_data = data["ship"]
    ship = Ship(ship_data["width"], ship_data["length"],
ship_data["max_capacity"])

    vehicles: dict[str, Vehicle] = {}
    for item in data["vehicles"]:
        vehicle = Vehicle(
            item["id"],
            item["width"],
            item["length"],
            item["weight"],
            item["shipping_fee"],
            item["eta"],
        )
        if vehicle.id in vehicles:
            raise ValueError(f"Duplicate vehicle ID: {vehicle.id}.")
        vehicles[vehicle.id] = vehicle

    return ship, vehicles

# Membaca data kapal dan kendaraan dari file JSON
def load_problem(path: str | Path) -> tuple[Ship, dict[str, Vehicle]]:
    data = json.loads(Path(path).read_text(encoding="utf-8"))
    return problem_from_data(data)

# Mengecek validasi penempatan kendaraan memenuhi batas ukuran deck, kapasitas,
dan tidak saling bertabrakan
def is_valid(state: State, ship: Ship, vehicles: dict[str, Vehicle]) -> bool:
    if state.placements.keys() != vehicles.keys():
        return False
```

```
    occupied: set[tuple[int, int]] = set()
    total_weight = 0.0

    for vehicle_id, placement in state.placements.items():
        if placement is None:
            continue

        vehicle = vehicles[vehicle_id]
        width, length = footprint(vehicle, placement.rotated)
        if placement.x < 0 or placement.y < 0 or placement.x + width > ship.width
or placement.y + length > ship.length:
            return False

        total_weight += vehicle.weight
        if total_weight > ship.max_capacity:
            return False

        for y in range(placement.y, placement.y + length):
            for x in range(placement.x, placement.x + width):
                if (x, y) in occupied:
                    return False
                occupied.add((x, y))

    return True

# Menghitung total shipping fee dari kendaraan yang dimuat ke kapal
def shipping_fee_fitness(state: State, vehicles: dict[str, Vehicle]) -> float:
    total_fee = 0
    for vehicle_id, placement in state.placements.items():
        if placement is not None:
            total_fee += vehicles[vehicle_id].shipping_fee

    return total_fee

# Menghitung skor prioritas kendaraan berdasarkan ETA, semakin kecil ETA semakin
besar skornya
def eta_fitness(state: State, vehicles: dict[str, Vehicle]) -> float:
    total_score = 0.0

    for vehicle_id, placement in state.placements.items():
        if placement is not None:
            vehicle = vehicles[vehicle_id]
            total_score += 1 / (vehicle.eta + 1)

    return total_score

# Mengubah state menjadi grid 2D untuk menampilkan posisi kendaraan pada deck
def grid_from_state(state: State, ship: Ship, vehicles: dict[str, Vehicle]) ->
list[list[str | None]]:
    if not is_valid(state, ship, vehicles):
        raise ValueError("State invalid")

    grid: list[list[str | None]] = []
    for y in range(ship.length):
        row: list[str | None] = []
        for x in range(ship.width):
```

```
            row.append(None)
        grid.append(row)

    for vehicle_id, placement in state.placements.items():
        if placement is None:
            continue

        vehicle = vehicles[vehicle_id]
        width, length = footprint(vehicle, placement.rotated)
        for y in range(placement.y, placement.y + length):
            for x in range(placement.x, placement.x + width):
                grid[y][x] = vehicle_id

    return grid

# Membuat state awal dengan mencoba menempatkan kendaraan pada posisi dan
orientasi secara acak
def random_state(ship: Ship, vehicles: dict[str, Vehicle], rng: random.Random) ->
State:
    state = State({vehicle_id: None for vehicle_id in vehicles})
    vehicle_ids = list(vehicles)
    rng.shuffle(vehicle_ids)

    for vehicle_id in vehicle_ids:
        vehicle = vehicles[vehicle_id]
        possible_placements: list[Placement] = []
        for rotated in (False, True):
            width, length = footprint(vehicle, rotated)
            for y in range(ship.length - length + 1):
                for x in range(ship.width - width + 1):
                    possible_placements.append(Placement(x, y, rotated))

        rng.shuffle(possible_placements)
        for placement in possible_placements:
            state.placements[vehicle_id] = placement
            if is_valid(state, ship, vehicles):
                break
            state.placements[vehicle_id] = None

    return state
```

Penjelasan neighbors.py

neighbors.py digunakan untuk menghasilkan neighbor, yaitu susunan kendaraan baru yang diperoleh dari perubahan pada state saat ini. Terdapat tiga operasi yang dapat dilakukan, yaitu move untuk memindahkan kendaraan, swap untuk menukar posisi dua kendaraan, dan rotate untuk mengubah orientasi kendaraan. Fungsi random_neighbor() digunakan untuk mencari satu neighbor secara acak, dan iter_neighbors() digunakan untuk mencari seluruh kemungkinan neighbor. Setiap hasil perubahan diperiksa menggunakan is_valid() agar tetap memenuhi batasan kapal.

```
import random
```

```
from collections.abc import Iterator

from .models import Placement, Ship, State, Vehicle
from .problem import footprint, is_valid

# Memindahkan kendaraan ke posisi baru atau mengeluarkannya dari kapal
def move(state: State, vehicle_id: str, placement: Placement | None, ship: Ship,
vehicles: dict[str, Vehicle]) -> State | None:
    if vehicle_id not in state.placements:
        return None

    current = state.placements[vehicle_id]
    if current is not None and placement is not None:
        placement = Placement(placement.x, placement.y, current.rotated)

    if current == placement:
        return None

    candidate = State(state.placements.copy())
    candidate.placements[vehicle_id] = placement

    if is_valid(candidate, ship, vehicles):
        return candidate

    return None

# Menukar posisi dua kendaraan
def swap(state: State, first_id: str, second_id: str, ship: Ship, vehicles:
dict[str, Vehicle]) -> State | None:
    if first_id not in state.placements or second_id not in state.placements:
        return None

    if state.placements[first_id] == state.placements[second_id]:
        return None

    candidate = State(state.placements.copy())
    candidate.placements[first_id] = state.placements[second_id]
    candidate.placements[second_id] = state.placements[first_id]

    if is_valid(candidate, ship, vehicles):
        return candidate

    return None

# Memutar kendaraan dengan posisi x dan y yang tetap
def rotate(state: State, vehicle_id: str, ship: Ship, vehicles: dict[str,
Vehicle]) -> State | None:
    placement = state.placements.get(vehicle_id)
    if placement is None:
        return None

    candidate = State(state.placements.copy())
    candidate.placements[vehicle_id] = Placement(placement.x, placement.y, not
placement.rotated)

    if is_valid(candidate, ship, vehicles):
        return candidate
```

```
    return None

# Mencari satu neighbor secara acak menggunakan move, swap, atau rotate
def random_neighbor(state: State, ship: Ship, vehicles: dict[str, Vehicle], rng:
random.Random, max_attempts: int = 100) -> State | None:
    vehicle_ids = list(vehicles)
    if not vehicle_ids:
        return None

    for _ in range(max_attempts):
        operation = rng.choice(("move", "swap", "rotate"))
        vehicle_id = rng.choice(vehicle_ids)

        if operation == "move":
            placement = None
            if rng.choice((True, False)):
                placement = Placement(rng.randrange(ship.width),
rng.randrange(ship.length), rng.choice((True, False)))
            candidate = move(state, vehicle_id, placement, ship, vehicles)
        elif operation == "swap":
            other_id = rng.choice(vehicle_ids)
            candidate = swap(state, vehicle_id, other_id, ship, vehicles)
        else:
            candidate = rotate(state, vehicle_id, ship, vehicles)

        if candidate is not None:
            return candidate

    return None

# Mencoba semua kemungkinan neighbor dari operasi move, swap, dan rotate
def iter_neighbors(state: State, ship: Ship, vehicles: dict[str, Vehicle]) ->
Iterator[State]:
    vehicle_ids = list(vehicles)
    seen = set()

    for index, vehicle_id in enumerate(vehicle_ids):
        current = state.placements[vehicle_id]

        if current is not None:
            candidate = move(state, vehicle_id, None, ship, vehicles)
            if candidate is not None:
                seen.add(tuple(candidate.placements.items()))
                yield candidate

        if current is None:
            orientations = (False, True)
        else:
            orientations = (current.rotated,)

        for rotated in orientations:
            width, length = footprint(vehicles[vehicle_id], rotated)
            for y in range(ship.length - length + 1):
                for x in range(ship.width - width + 1):
                    placement = Placement(x, y, rotated)
                    candidate = move(state, vehicle_id, placement, ship, vehicles)
```

```
                if candidate is None:
                    continue

                key = tuple(candidate.placements.items())
                if key in seen:
                    continue

                seen.add(key)
                yield candidate

        for other_id in vehicle_ids[index + 1:]:
            candidate = swap(state, vehicle_id, other_id, ship, vehicles)
            if candidate is None:
                continue

            key = tuple(candidate.placements.items())
            if key in seen:
                continue

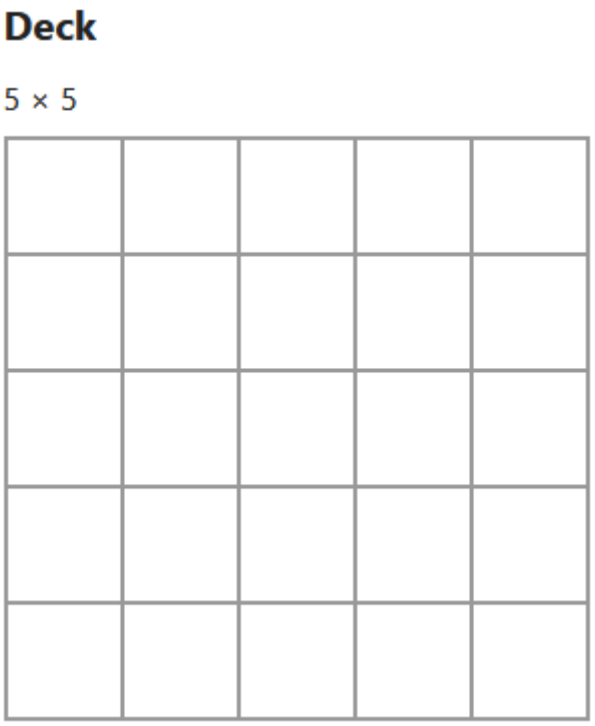
            seen.add(key)
            yield candidate

        if current is not None:
            candidate = rotate(state, vehicle_id, ship, vehicles)
            if candidate is not None:
                key = tuple(candidate.placements.items())
                if key not in seen:
                    seen.add(key)
                    yield candidate
```

Insialisasi State

Percobaan dilakukan menggunakan test case example.json yang berisi sebuah ship dengan ukuran grid 5 × 5 dan 5 vehicles dengan karakteristik yang bervariasi. Seluruh algoritma dijalankan menggunakan seed yang berbeda tiap percobaan, yaitu 3070, 3071, 3072, untuk menjaga konsistensi pengujian. Objective dari percobaan ini adalah memaksimalkan total shipping fee. Global optimum untuk test case ini adalah shipping fee 140 dan ETA 1.283333332 yang diperoleh melalui algoritma brute force.

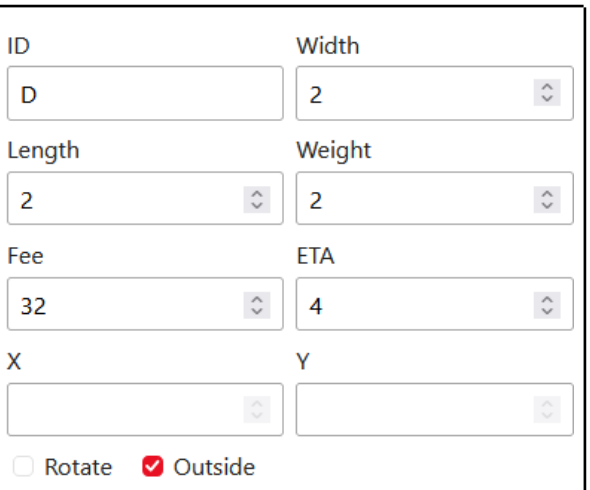
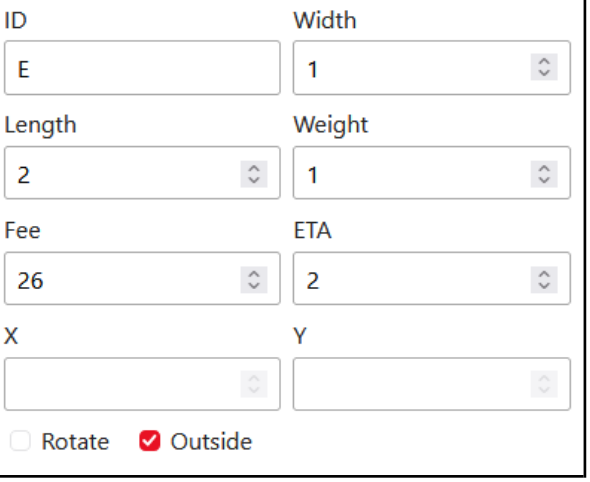
Berikut adalah gambar **state awal** untuk seluruh algoritma.



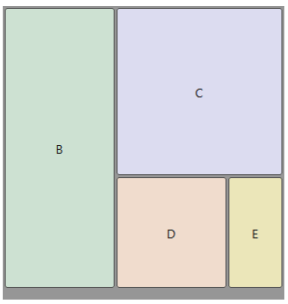
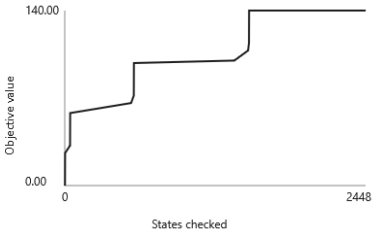
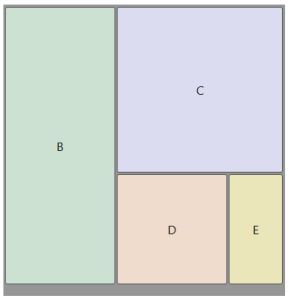
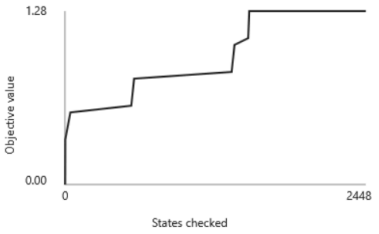
Berikut merupakan detail dari ship dan vehicles.

Spesifikasi Ship	**Ship** Width: 5 Length: 5 Maximum capacity: 12

Vehicle A	ID: A Width: 3 Length: 5 Weight: 7 Fee: 74 ETA: 2 X: Y: ☐ Rotate ☑ Outside
Vehicle B	B 2 Length: 5 Weight: 4 Fee: 42 ETA: 1 X: Y: ☐ Rotate ☑ Outside
Vehicle C	ID: C Width: 3 Length: 3 Weight: 5 Fee: 40 ETA: 3 X: Y: ☐ Rotate ☑ Outside

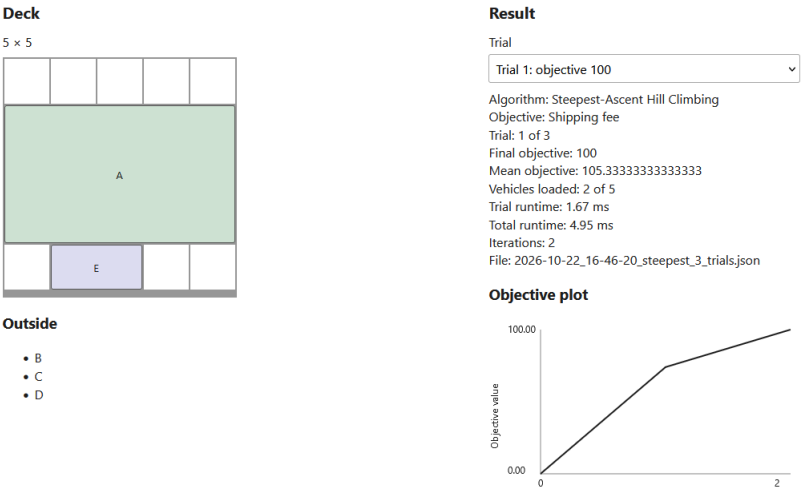
Vehicle D	
Vehicle E	

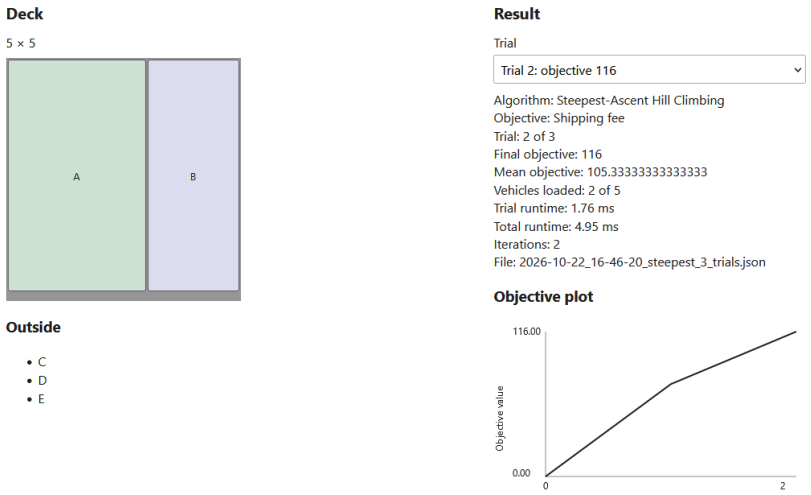
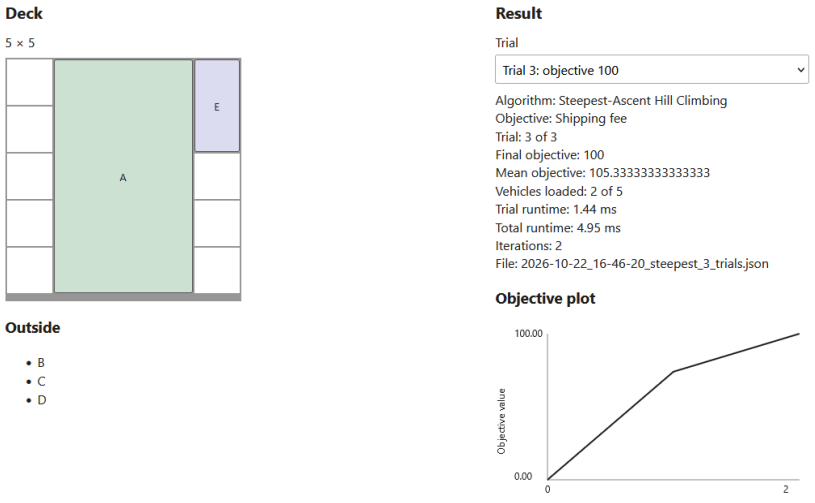
Berikut adalah hasil dari Brute Force untuk mendapatkan global optimum

Shipping Fee	**Deck** 5 × 5  **Outside** • A **Result** Trial Trial 1: objective 140 Algorithm: Brute Force Objective: Shipping fee Trial: 1 of 3 Final objective: 140 Mean objective: 140 Vehicles loaded: 4 of 5 Trial runtime: 37.65 ms Total runtime: 112.99 ms Complete states checked: 2448 File: 2026-10-22_17-25-43_brute_force_3_trials.json **Objective plot** 
ETA	**Deck** 5 × 5  **Outside** • A **Result** Trial Trial 1: objective 1.2833333333333332 Algorithm: Brute Force Objective: ETA Trial: 1 of 3 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 29.51 ms Total runtime: 87.88 ms Complete states checked: 2448 File: 2026-10-22_19-39-02_brute_force_3_trials.json **Objective plot** 

Hasil Eksperimen (Nilai Objective: Shipping Fee)

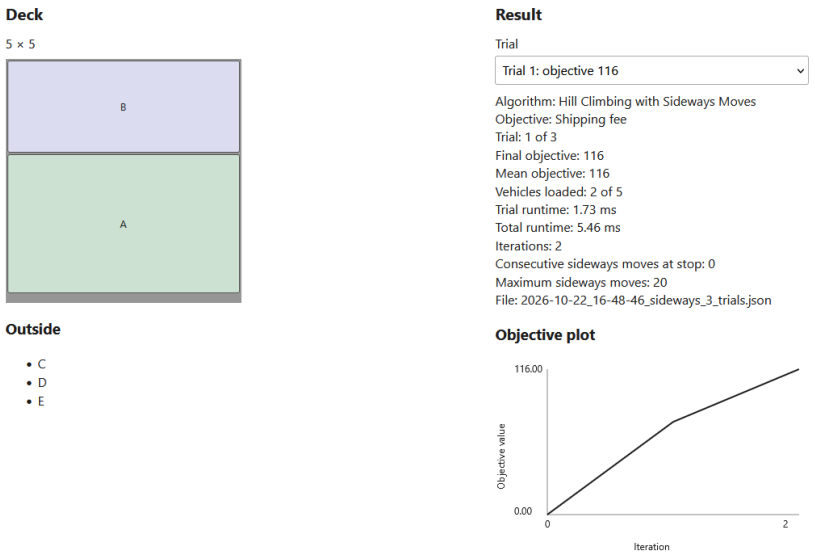
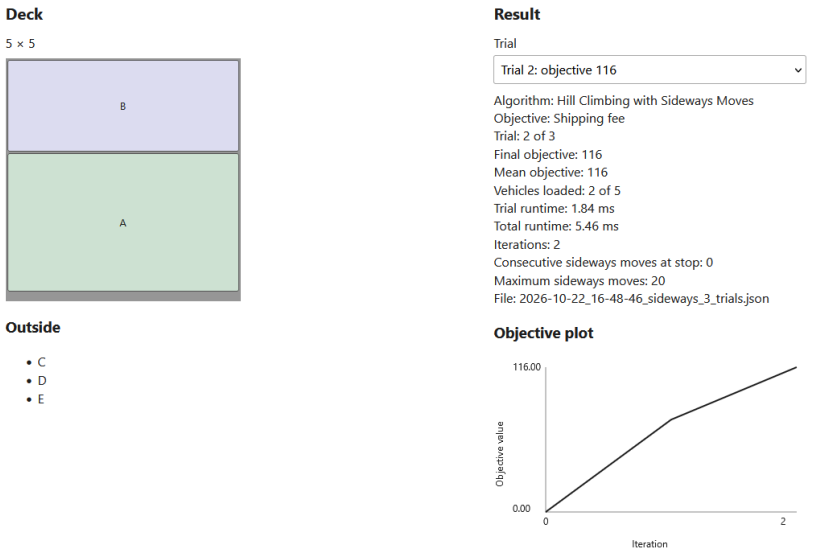
Steepest Ascent Hill Climbing

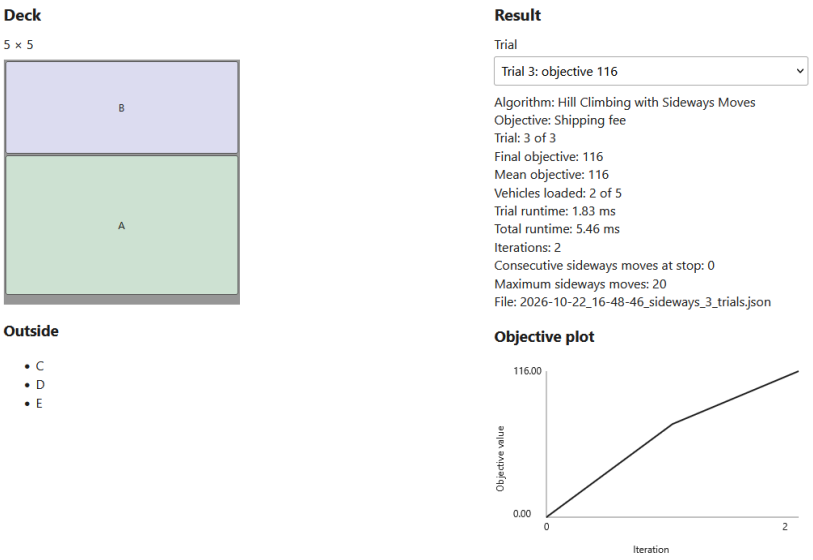
<table>
<tr><td rowspan="2">Percobaan ke-1</td><td>Ringkasan Hasil</td><td>Objective: Shipping fee
Trial: 1 of 3
Final objective: 100
Mean objective: 105.33333333333333
Vehicles loaded: 2 of 5
Trial runtime: 1.67 ms
Total runtime: 4.95 ms
Iterations: 2</td></tr>
<tr><td>Visualisasi</td><td> 
</td></tr>
<tr><td>Percobaan ke-2</td><td>Ringkasan Hasil</td><td>Objective: Shipping fee
Trial: 2 of 3
Final objective: 116
Mean objective: 105.33333333333333
Vehicles loaded: 2 of 5
Trial runtime: 1.76 ms
Total runtime: 4.95 ms
Iterations: 2</td></tr>
</table>

	Visualisasi	
Percobaan ke-3	Ringkasan Hasil	Objective: Shipping fee Trial: 3 of 3 Final objective: 100 Mean objective: 105.33333333333333 Vehicles loaded: 2 of 5 Trial runtime: 1.44 ms Total runtime: 4.95 ms Iterations: 2
	Visualisasi	

Sideways Move Hill Climbing

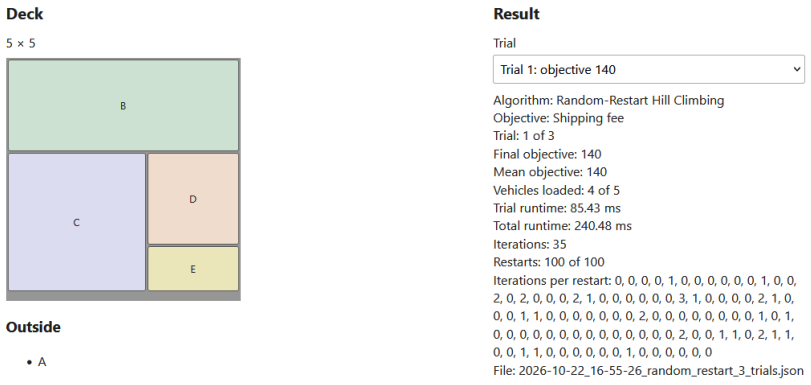
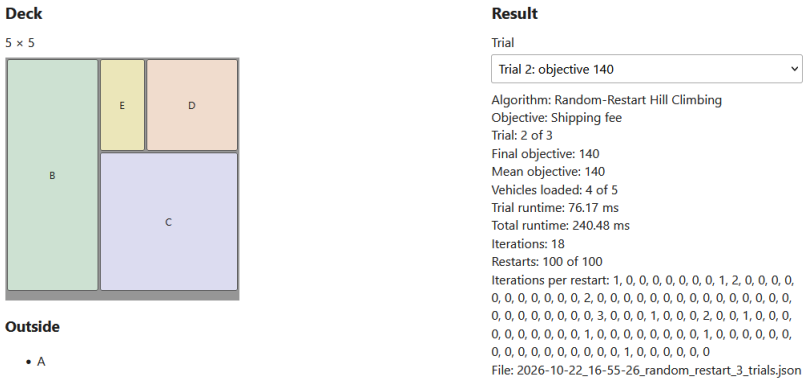
Percobaan ke-1	Ringkasan Hasil	Objective: Shipping fee Trial: 1 of 3 Final objective: 116 Mean objective: 116 Vehicles loaded: 2 of 5 Trial runtime: 1.73 ms

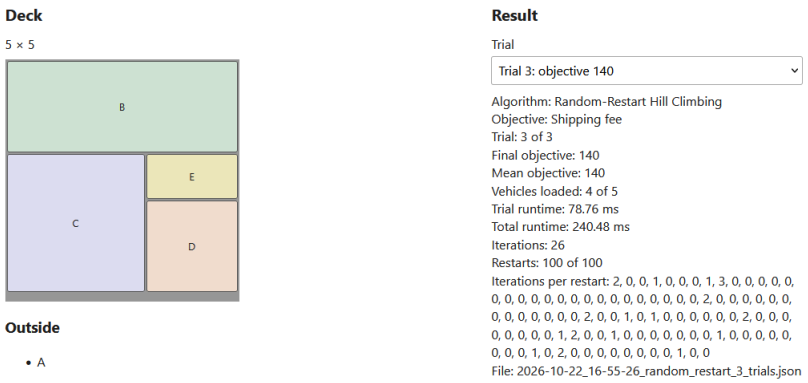
		Total runtime: 5.46 ms Iterations: 2 Consecutive sideways moves at stop: 0 Maximum sideways moves: 20
	Visualisasi	
Percobaan ke-2	Ringkasan Hasil	Objective: Shipping fee Trial: 2 of 3 Final objective: 116 Mean objective: 116 Vehicles loaded: 2 of 5 Trial runtime: 1.84 ms Total runtime: 5.46 ms Iterations: 2 Consecutive sideways moves at stop: 0 Maximum sideways moves: 20
	Visualisasi	
Percobaan	Ringkasan	Objective: Shipping fee

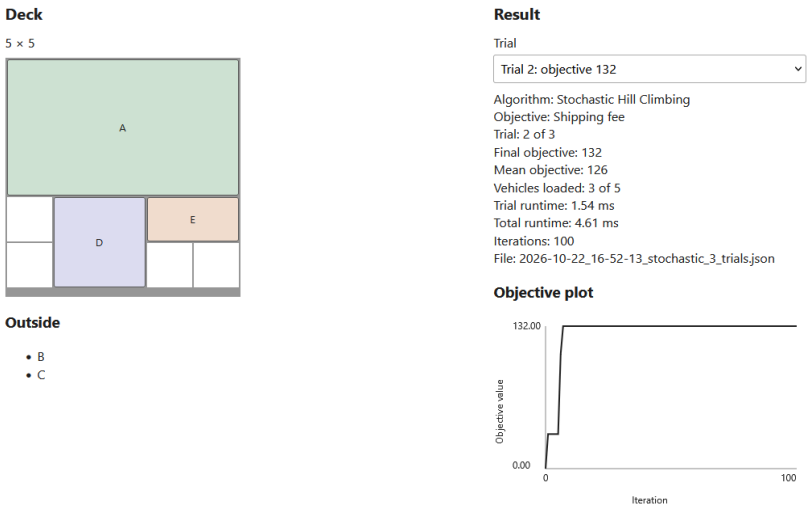
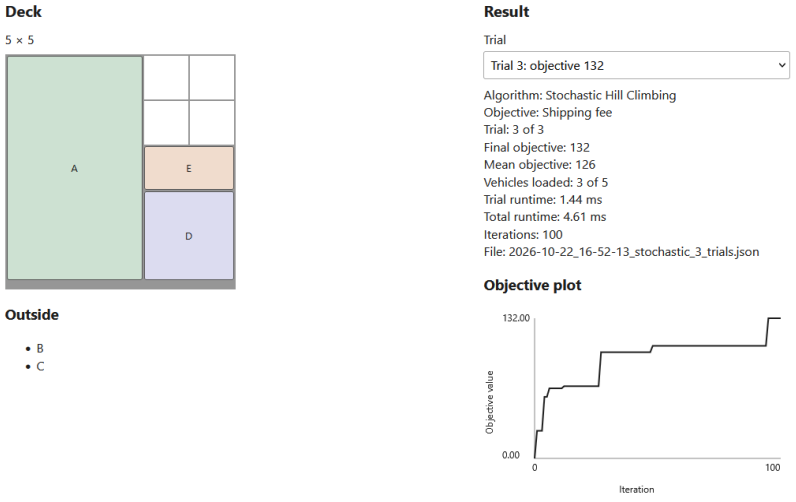
ke-3	Hasil	Trial: 3 of 3 Final objective: 116 Mean objective: 116 Vehicles loaded: 2 of 5 Trial runtime: 1.83 ms Total runtime: 5.46 ms Iterations: 2 Consecutive sideways moves at stop: 0 Maximum sideways moves: 20
	Visualisasi	

Random Restart Hill Climbing

Percobaan ke-1	Ringkasan Hasil	Objective: Shipping fee Trial: 1 of 3 Final objective: 140 Mean objective: 140 Vehicles loaded: 4 of 5 Trial runtime: 85.43 ms Total runtime: 240.48 ms Iterations: 35 Restarts: 100 of 100 Iterations per restart: 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 2, 0, 2, 0, 0, 0, 2, 1, 0, 0, 0, 0, 0, 0, 3, 1, 0, 0, 0, 0, 2, 1, 0, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 1, 1, 0, 2, 1, 1, 0, 0, 1, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0

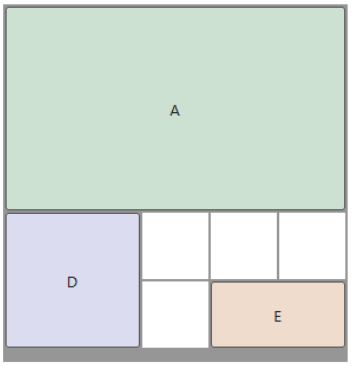
	Visualisasi	
Percobaan ke-2	Ringkasan Hasil	Objective: Shipping fee Trial: 2 of 3 Final objective: 140 Mean objective: 140 Vehicles loaded: 4 of 5 Trial runtime: 76.17 ms Total runtime: 240.48 ms Iterations: 18 Restarts: 100 of 100 Iterations per restart: 1, 0, 0, 0, 0, 0, 0, 0, 1, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 3, 0, 0, 0, 1, 0, 0, 0, 2, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0
	Visualisasi	
Percobaan ke-3	Ringkasan Hasil	Objective: Shipping fee Trial: 3 of 3 Final objective: 140 Mean objective: 140 Vehicles loaded: 4 of 5 Trial runtime: 78.76 ms Total runtime: 240.48 ms Iterations: 26 Restarts: 100 of 100 Iterations per restart: 2, 0, 0, 1, 0, 0, 0, 1, 3, 0, 0, 0, 0, 0, 0, 0,

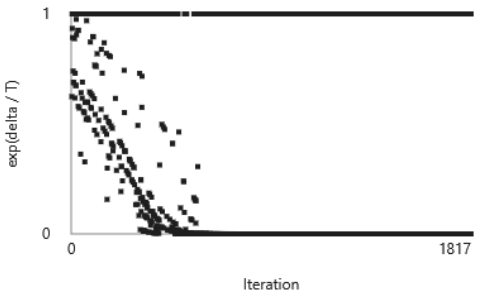
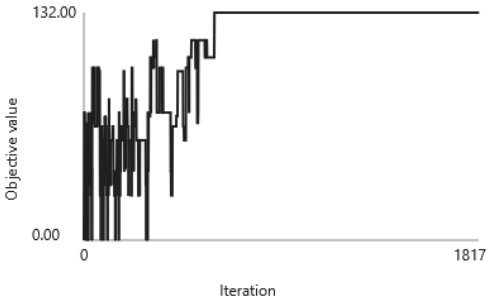
		0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 1, 0, 1, 0, 0, 0, 0, 0, 0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 1, 2, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 2, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0
	Visualisasi	 Deck 5 × 5 B E C D Outside • A Result Trial Trial 3: objective 140 Algorithm: Random-Restart Hill Climbing Objective: Shipping fee Trial: 3 of 3 Final objective: 140 Mean objective: 140 Vehicles loaded: 4 of 5 Trial runtime: 78.76 ms Total runtime: 240.48 ms Iterations: 26 Restarts: 100 of 100 Iterations per restart: 2, 0, 0, 1, 0, 0, 0, 1, 3, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0

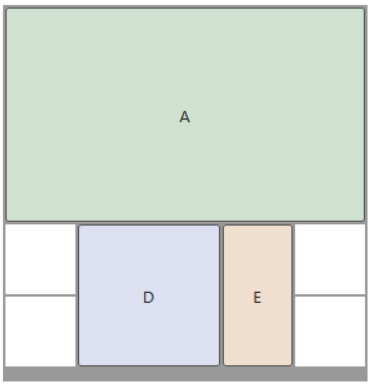
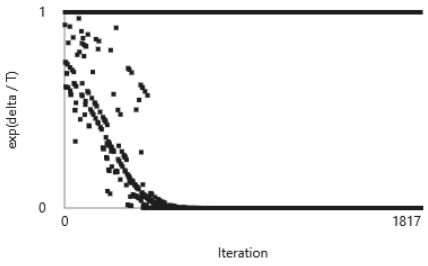
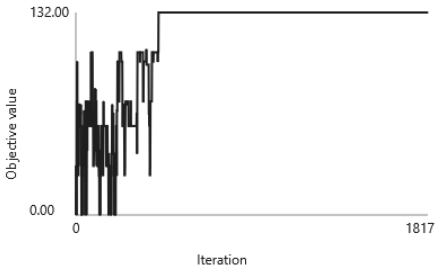
		Trial runtime: 1.54 ms Total runtime: 4.61 ms Iterations: 100
	Visualisasi	
Percobaan ke-3	Ringkasan Hasil	Objective: Shipping fee Trial: 3 of 3 Final objective: 132 Mean objective: 126 Vehicles loaded: 3 of 5 Trial runtime: 1.44 ms Total runtime: 4.61 ms Iterations: 100
	Visualisasi	

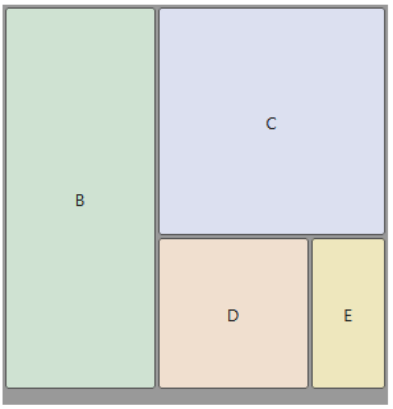
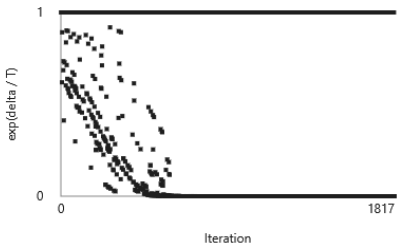
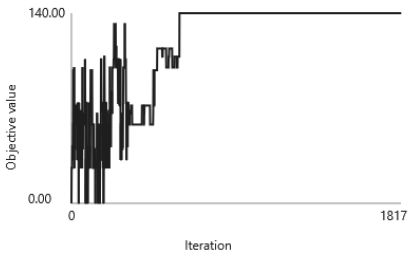
Simulated Annealing

Percobaan	Ringkasan	Objective: Shipping fee

ke-1	Hasil	Trial: 1 of 3 Final objective: 132 Mean objective: 134.66666666666666 Vehicles loaded: 3 of 5 Trial runtime: 70.43 ms Total runtime: 215.42 ms Iterations: 1817 Stuck at local optimum: 1327 iterations (73.0%)
	Visualisasi	**Deck** 5 × 5  **Outside** • B • C

		Algorithm: Simulated Annealing Objective: Shipping fee Trial: 1 of 3 Final objective: 132 Mean objective: 134.66666666666666 Vehicles loaded: 3 of 5 Trial runtime: 70.43 ms Total runtime: 215.42 ms Iterations: 1817 Stuck at local optimum: 1327 iterations (73.0%) File: 2026-10-22_17-20-03_annealing_3_trials.json **e^(delta / T) plot by iteration**  **Objective plot** 
Percobaan ke-2	Ringkasan Hasil	Objective: Shipping fee Trial: 2 of 3 Final objective: 132 Mean objective: 134.66666666666666 Vehicles loaded: 3 of 5 Trial runtime: 69.68 ms Total runtime: 215.42 ms Iterations: 1817 Stuck at local optimum: 1398 iterations (76.9%)

	Visualisasi	**Deck** 5 × 5  **Outside** • B • C Algorithm: Simulated Annealing Objective: Shipping fee Trial: 2 of 3 Final objective: 132 Mean objective: 134.66666666666666 Vehicles loaded: 3 of 5 Trial runtime: 69.68 ms Total runtime: 215.42 ms Iterations: 1817 Stuck at local optimum: 1398 iterations (76.9%) File: 2026-10-22_17-20-03_annealing_3_trials.json **e^(delta / T) plot by iteration**  **Objective plot** 
Percobaan ke-3	Ringkasan Hasil	Objective: Shipping fee Trial: 3 of 3 Final objective: 140 Mean objective: 134.66666666666666 Vehicles loaded: 4 of 5

		Trial runtime: 75.20 ms Total runtime: 215.42 ms Iterations: 1817 Stuck at local optimum: 1269 iterations (69.8%)
	Visualisasi	**Deck** 5 × 5  **Outside** • A Trial Trial 3: objective 140 Algorithm: Simulated Annealing Objective: Shipping fee Trial: 3 of 3 Final objective: 140 Mean objective: 134.66666666666666 Vehicles loaded: 4 of 5 Trial runtime: 75.20 ms Total runtime: 215.42 ms Iterations: 1817 Stuck at local optimum: 1269 iterations (69.8%) File: 2026-10-22_17-20-03_annealing_3_trials.json **e^(delta / T) plot by iteration**  **Objective plot** 

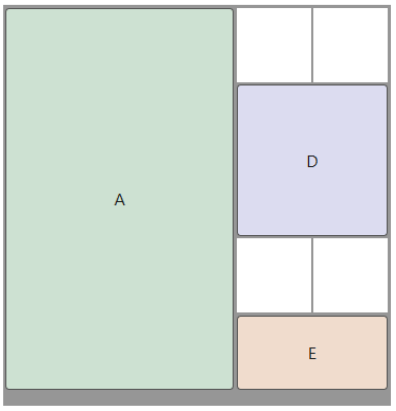
Genetic Algorithm

Pada grafik hasil Genetic Algorithm, garis penuh (solid) menunjukkan nilai fitness tertinggi pada setiap generasi, sedangkan garis putus-putus (dashed) menunjukkan rata-rata fitness populasi. Nilai fitness maksimum dapat mengalami kenaikan maupun penurunan karena populasi diganti pada setiap generasi.

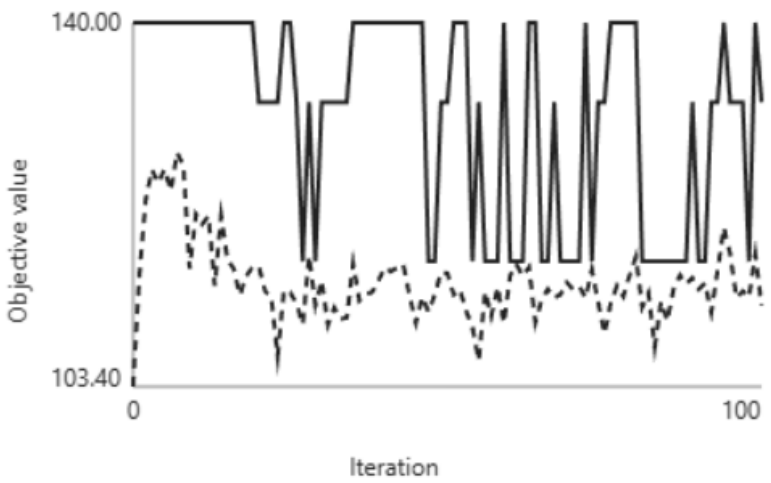
Setiap konfigurasi dijalankan sebanyak tiga kali secara terpisah menggunakan seed 3070, 3071, dan 3072. Karena setiap eksekusi hanya menjalankan satu percobaan, hasil program menampilkan Trial: 1 of 1.

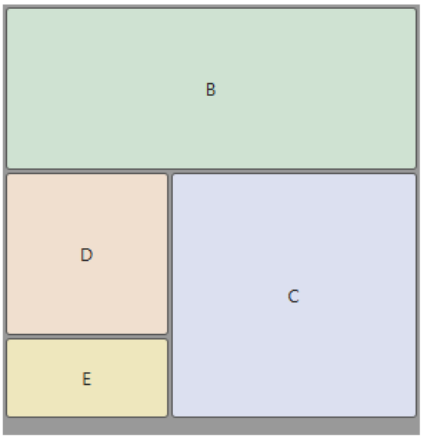
Jumlah Populasi Sebagai Kontrol (Populasi = 30)

Variasi 1: Iterasi = 100

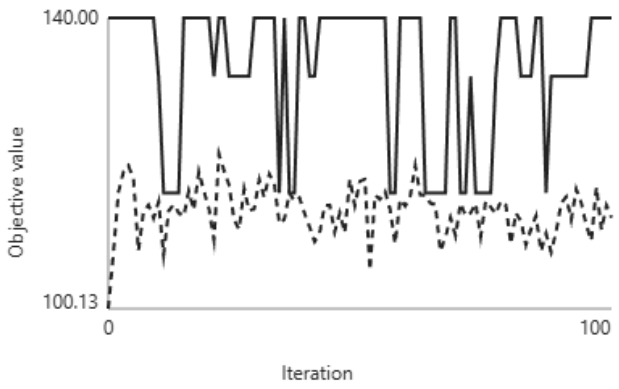
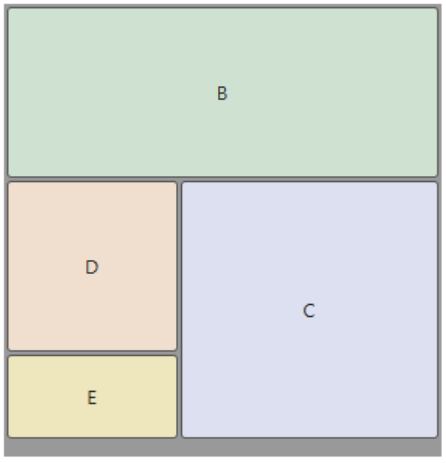
<table>
<tr><td rowspan="2">Percobaan ke-1</td><td>Ringkasan Hasil</td><td>Objective: Shipping fee
Trial: 1 of 1
Final objective: 132
Mean objective: 132
Vehicles loaded: 3 of 5
Trial runtime: 2203.83 ms
Total runtime: 2203.88 ms
Iterations: 100
Population: 30</td></tr>
<tr><td>Visualisasi</td><td>Deck
5 × 5
 

Outside
• B
• C</td></tr>
</table>

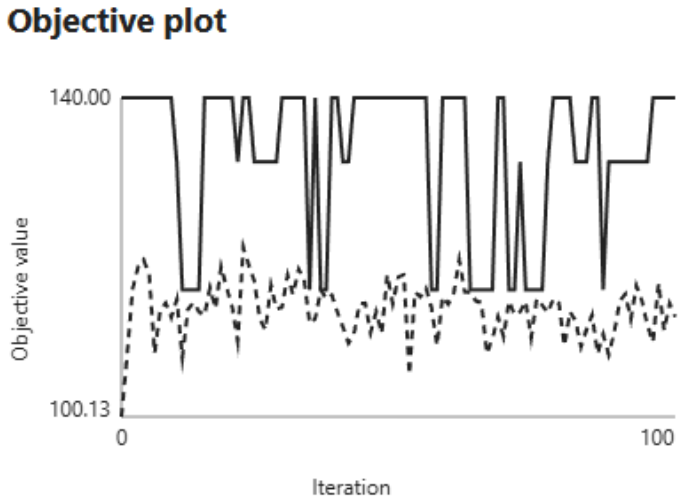
<table>
<tr><td></td><td></td><td>Objective plot
 

Solid = max; dashed = average</td></tr>
<tr><td rowspan="2">Percobaan ke-2</td><td>Ringkasan Hasil</td><td>Objective: Shipping fee
Trial: 1 of 1
Final objective: 140
Mean objective: 140
Vehicles loaded: 4 of 5
Trial runtime: 3622.18 ms
Total runtime: 3622.25 ms
Iterations: 100
Population: 30</td></tr>
<tr><td>Visualisasi</td><td>Deck
5 × 5
 

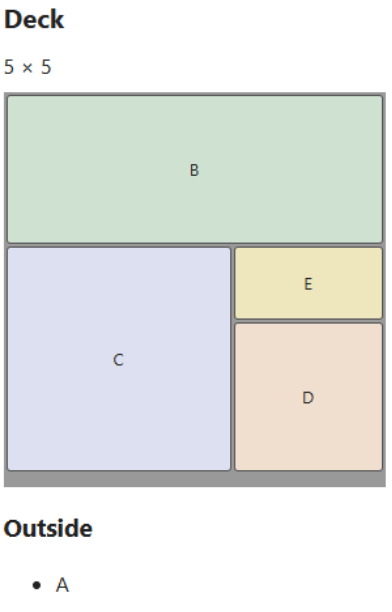
Outside
• A</td></tr>
</table>

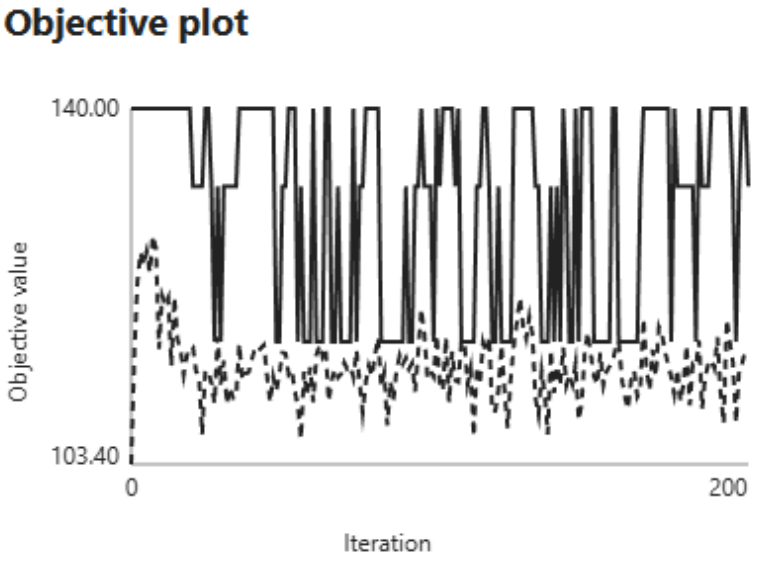
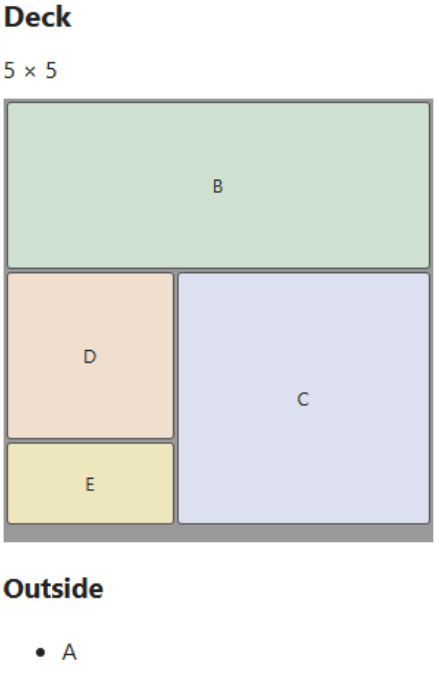
<table>
<tr><td></td><td></td><td>Objective plot
 
</td></tr>
<tr><td rowspan="2">Percobaan ke-3</td><td>Ringkasan Hasil</td><td>Objective: Shipping fee
Trial: 1 of 1
Final objective: 140
Mean objective: 140
Vehicles loaded: 4 of 5
Trial runtime: 3622.18 ms
Total runtime: 3622.25 ms
Iterations: 100
Population: 30</td></tr>
<tr><td>Visualisasi</td><td>Deck
5 × 5
 

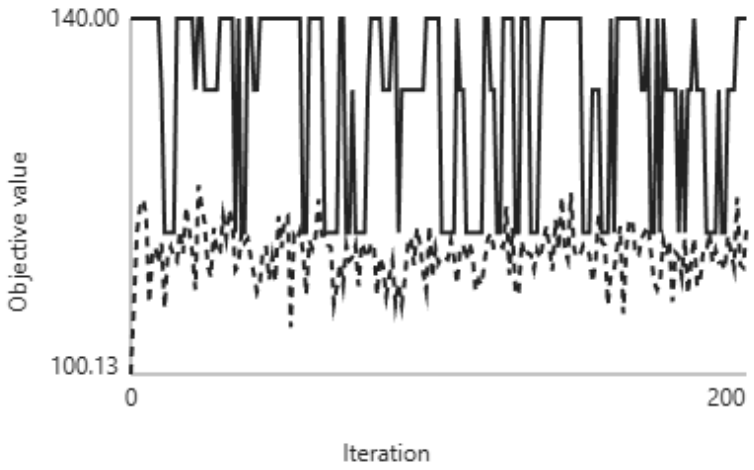
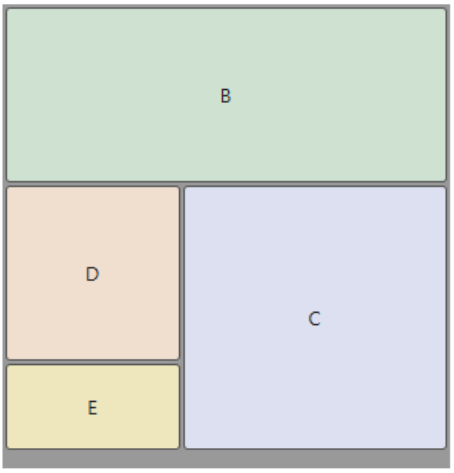
Outside
• A</td></tr>
</table>

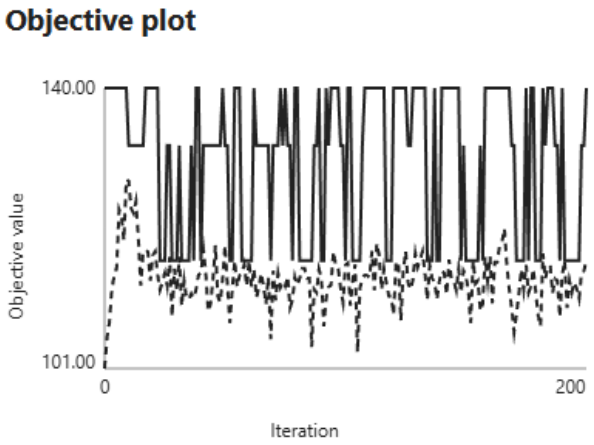
Variasi 2: Iterasi = 200

Percobaan ke-1	Ringkasan Hasil	Objective: Shipping fee Trial: 1 of 1 Final objective: 140 Mean objective: 140 Vehicles loaded: 4 of 5 Trial runtime: 9809.18 ms Total runtime: 9809.29 ms Iterations: 200 Population: 30
	Visualisasi	

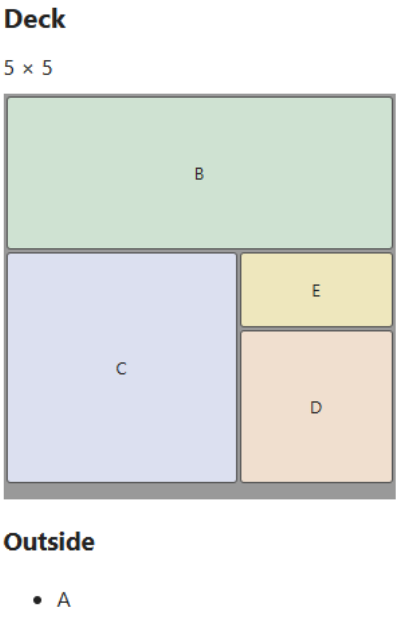
		
Percobaan ke-2	Ringkasan Hasil	Objective: Shipping fee Trial: 1 of 1 Final objective: 140 Mean objective: 140 Vehicles loaded: 4 of 5 Trial runtime: 7512.97 ms Total runtime: 7513.08 ms Iterations: 200 Population: 30
	Visualisasi	

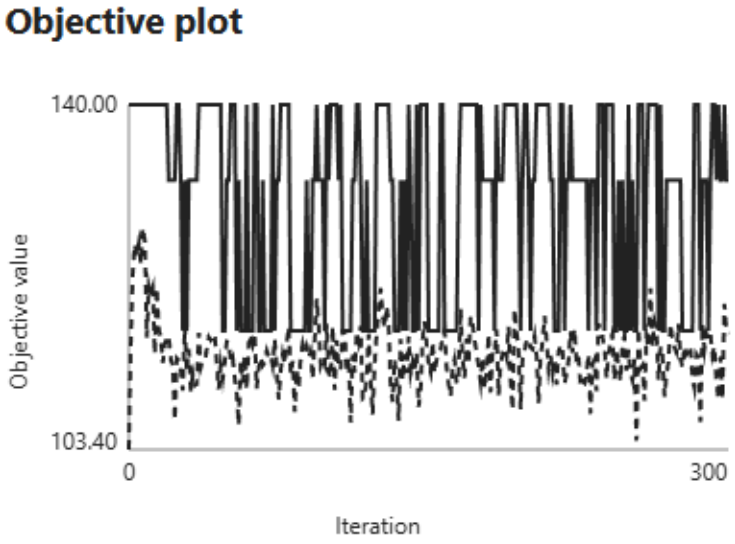
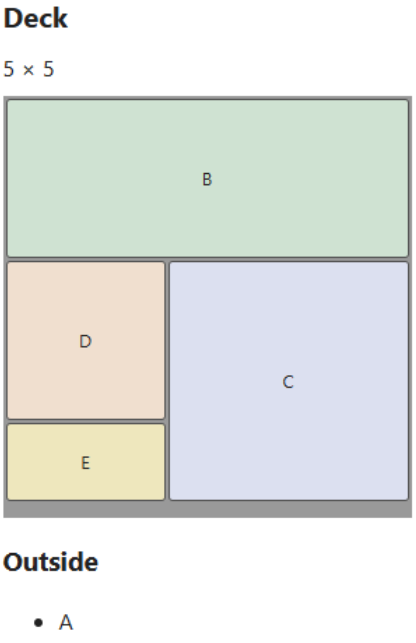
<table>
<tr><td></td><td></td><td>Objective plot
 
</td></tr>
<tr><td rowspan="2">Percobaan ke-3</td><td>Ringkasan Hasil</td><td>Algorithm: Genetic Algorithm
Objective: Shipping fee
Trial: 1 of 1
Final objective: 140
Mean objective: 140
Vehicles loaded: 4 of 5
Trial runtime: 11956.10 ms
Total runtime: 11956.20 ms
Iterations: 200
Population: 30</td></tr>
<tr><td>Visualisasi</td><td>Deck
5 × 5
 

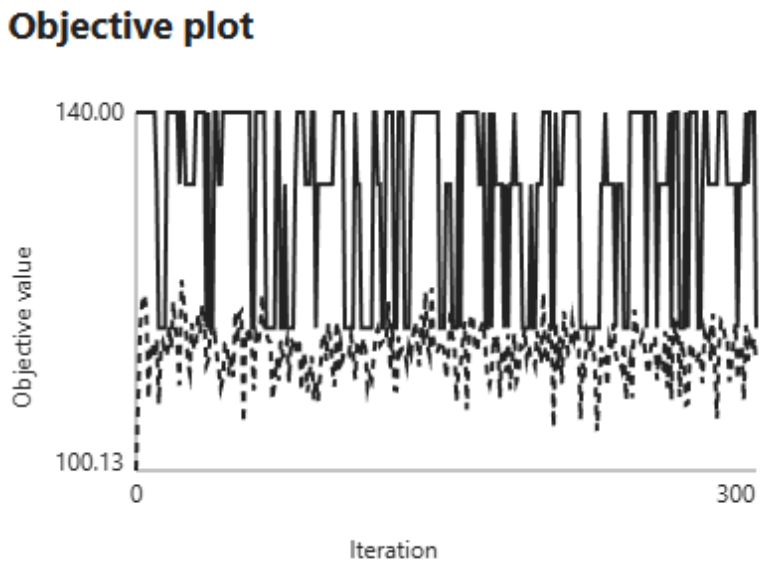
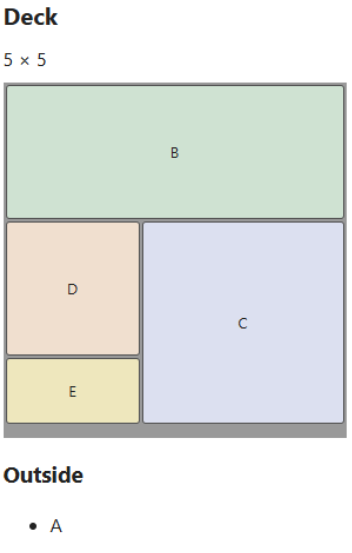
Outside
• A</td></tr>
</table>

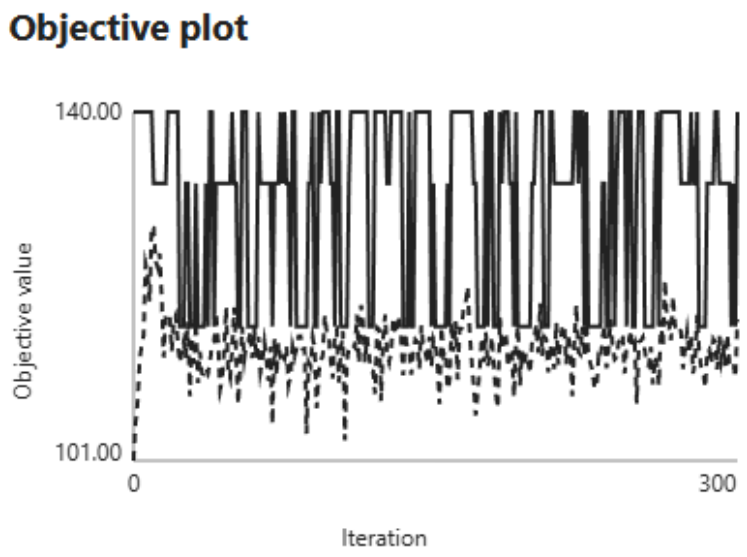
		

Variasi 3: Iterasi = 300

Percobaan ke-1	Ringkasan Hasil	Objective: Shipping fee Trial: 1 of 1 Final objective: 140 Mean objective: 140 Vehicles loaded: 4 of 5 Trial runtime: 10637.63 ms Total runtime: 10637.75 ms Iterations: 300 Population: 30
	Visualisasi	

		
Percobaan ke-2	Ringkasan Hasil	Objective: Shipping fee Trial: 1 of 1 Final objective: 140 Mean objective: 140 Vehicles loaded: 4 of 5 Trial runtime: 22097.10 ms Total runtime: 22097.27 ms Iterations: 300 Population: 30
	Visualisasi	

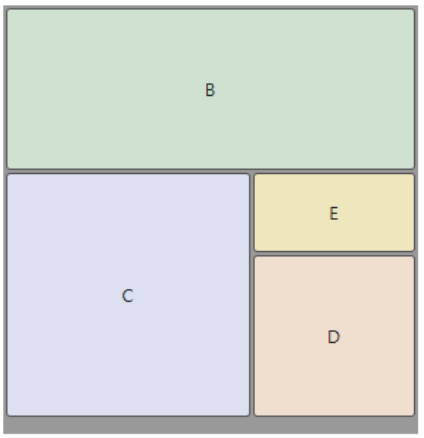
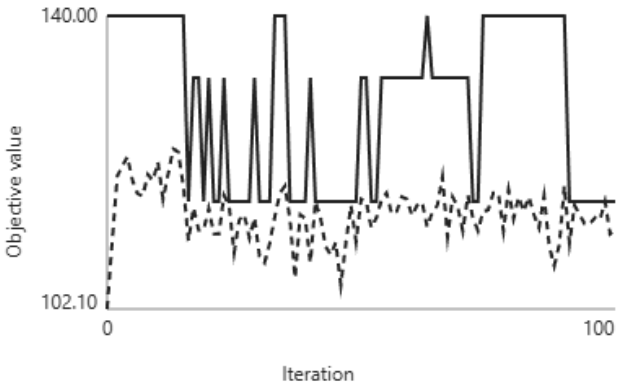
<table>
<tr><td></td><td></td><td> 
</td></tr>
<tr><td rowspan="2">Percobaan ke-3</td><td>Ringkasan Hasil</td><td>Objective: Shipping fee
Trial: 1 of 1
Final objective: 140
Mean objective: 140
Vehicles loaded: 4 of 5
Trial runtime: 11356.21 ms
Total runtime: 11356.31 ms
Iterations: 300
Population: 30</td></tr>
<tr><td>Visualisasi</td><td> 
</td></tr>
</table>

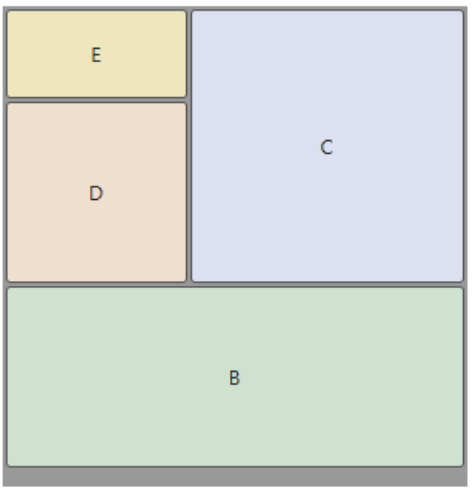
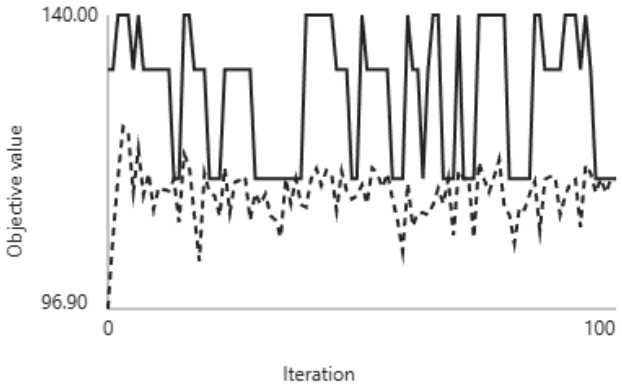
		

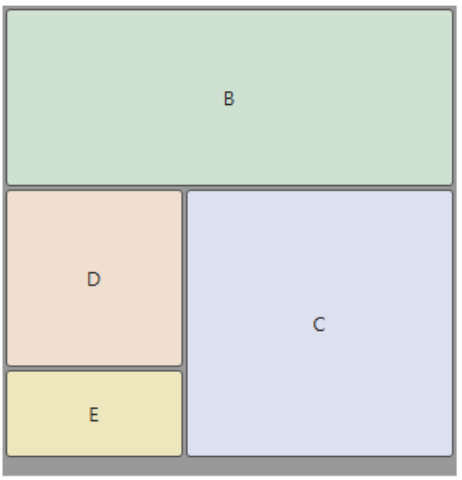
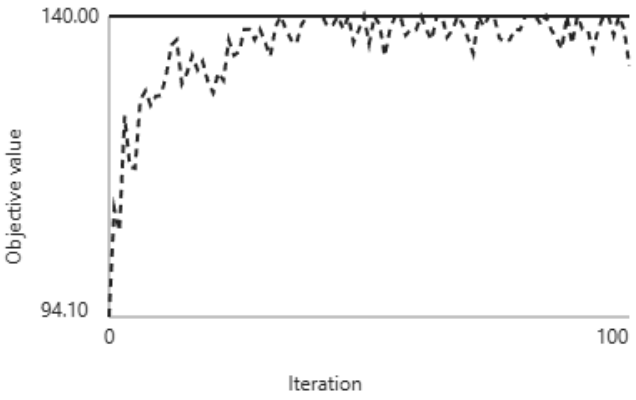
Jumlah Iterasi Sebagai Kontrol (Iterasi = 100)

Variasi 1: Populasi = 20

Percobaan ke-1	Ringkasan Hasil	Objective: Shipping fee Trial: 1 of 1 Final objective: 140 Mean objective: 140 Vehicles loaded: 4 of 5 Trial runtime: 1786.86 ms Total runtime: 1786.93 ms Iterations: 100 Population: 20

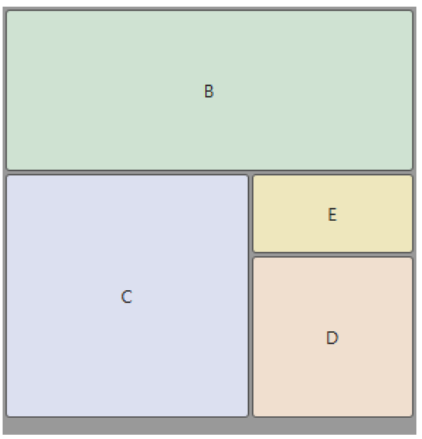
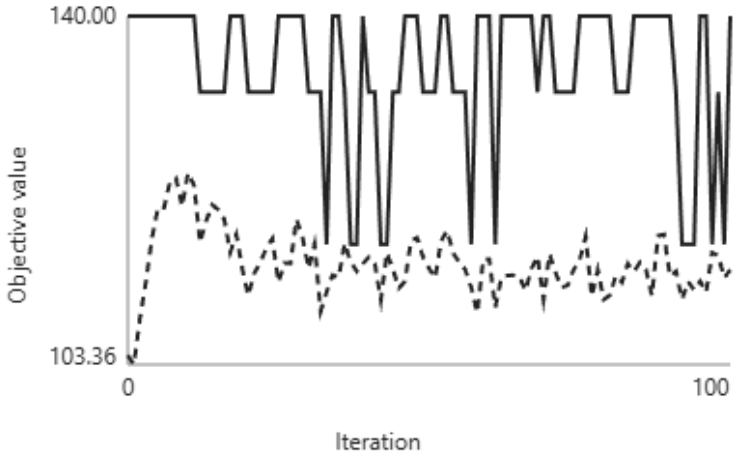
	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot**  Solid = max; dashed = average
Percobaan ke-2	Ringkasan Hasil	Objective: Shipping fee Trial: 1 of 1 Final objective: 140 Mean objective: 140 Vehicles loaded: 4 of 5 Trial runtime: 2321.02 ms Total runtime: 2321.10 ms Iterations: 100 Population: 20

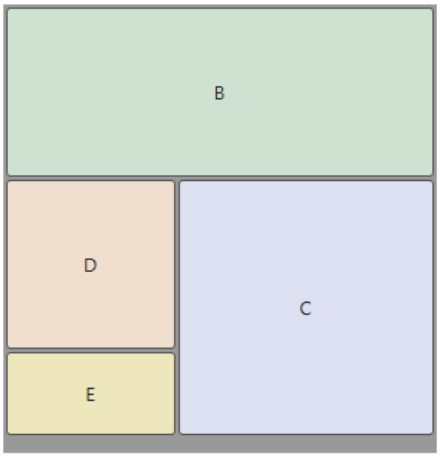
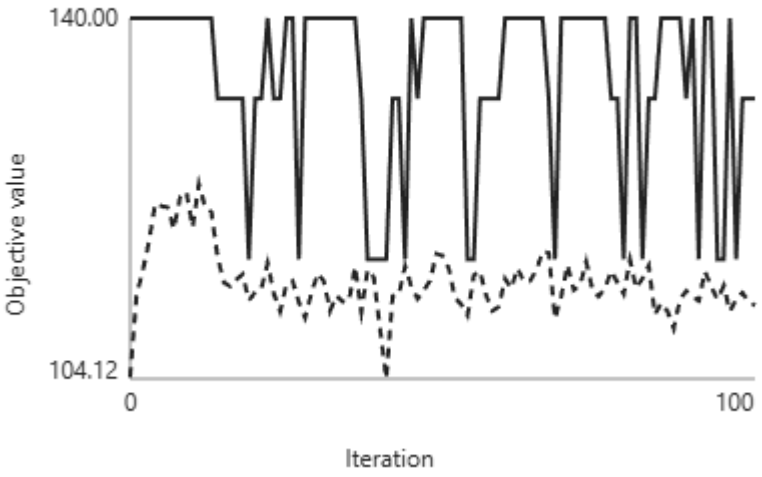
	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot**  Solid = max; dashed = average
Percobaan ke-3	Ringkasan Hasil	Objective: Shipping fee Trial: 1 of 1 Final objective: 140 Mean objective: 140 Vehicles loaded: 4 of 5 Trial runtime: 876.69 ms Total runtime: 876.77 ms Iterations: 100 Population: 20

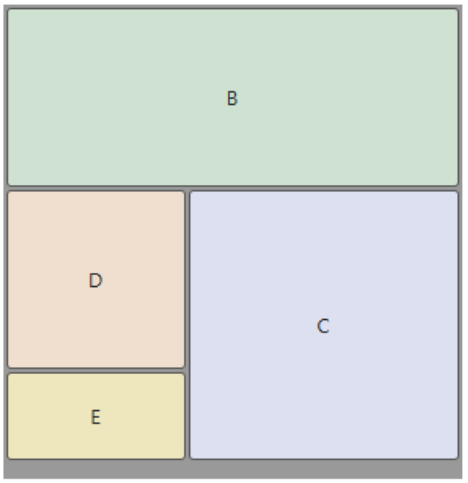
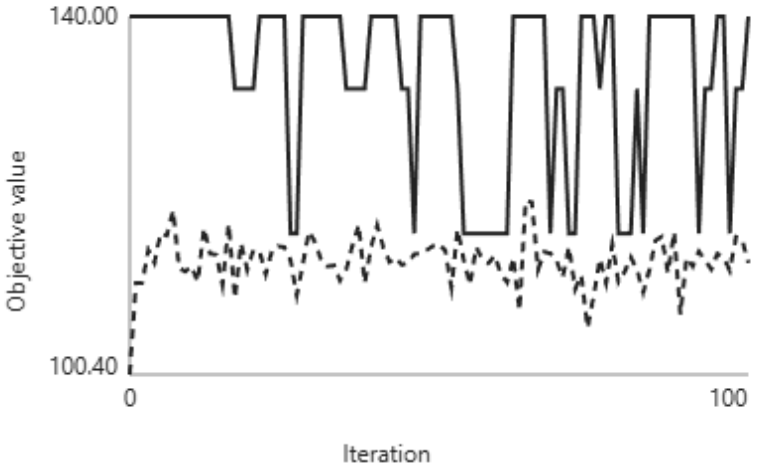
	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot**  Solid = max; dashed = average

Variasi 2: Populasi = 50

Percobaan ke-1	Ringkasan Hasil	Objective: Shipping fee Trial: 1 of 1 Final objective: 140 Mean objective: 140 Vehicles loaded: 4 of 5 Trial runtime: 7114.44 ms Total runtime: 7114.62 ms Iterations: 100 Population: 50

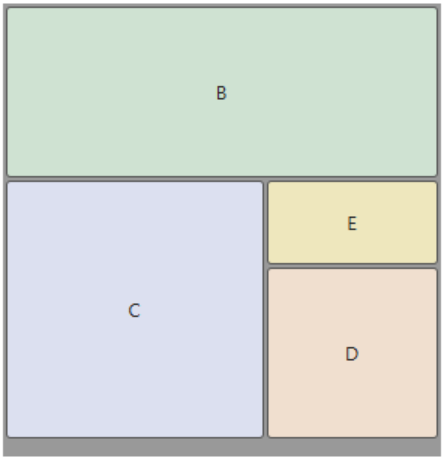
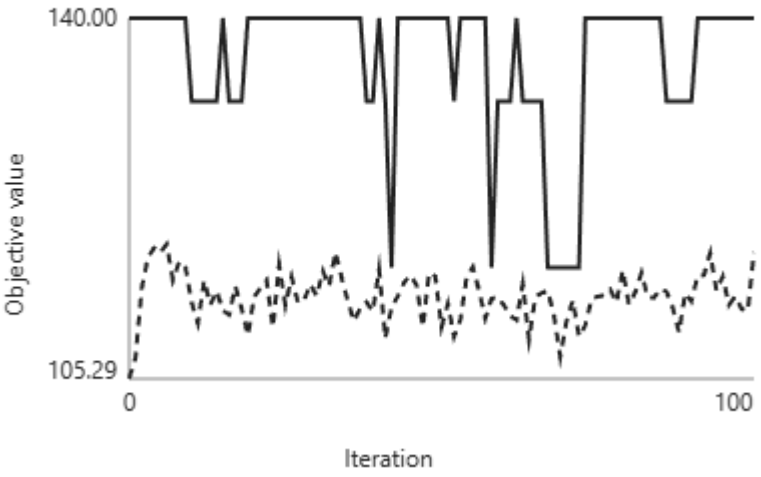
	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot** 
Percobaan ke-2	Ringkasan Hasil	Objective: Shipping fee Trial: 1 of 1 Final objective: 140 Mean objective: 140 Vehicles loaded: 4 of 5 Trial runtime: 7110.67 ms Total runtime: 7110.76 ms Iterations: 100 Population: 50

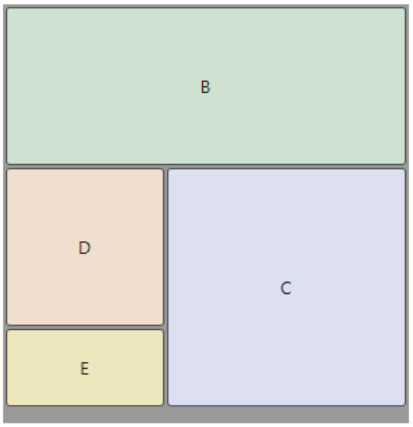
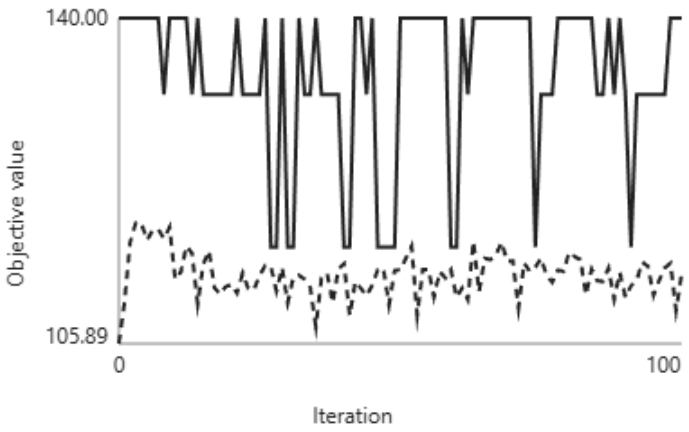
	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot** 
Percobaan ke-3	Ringkasan Hasil	Objective: Shipping fee Trial: 1 of 1 Final objective: 140 Mean objective: 140 Vehicles loaded: 4 of 5 Trial runtime: 9033.48 ms Total runtime: 9033.58 ms Iterations: 100 Population: 50

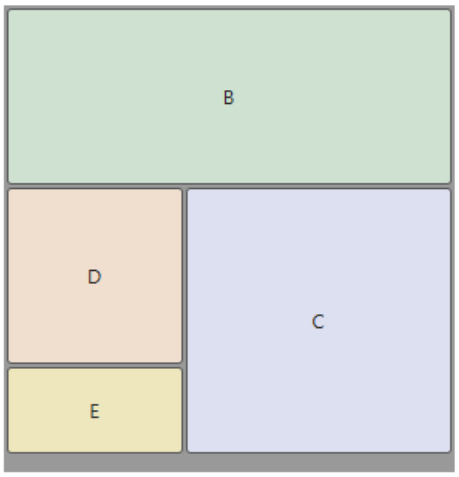
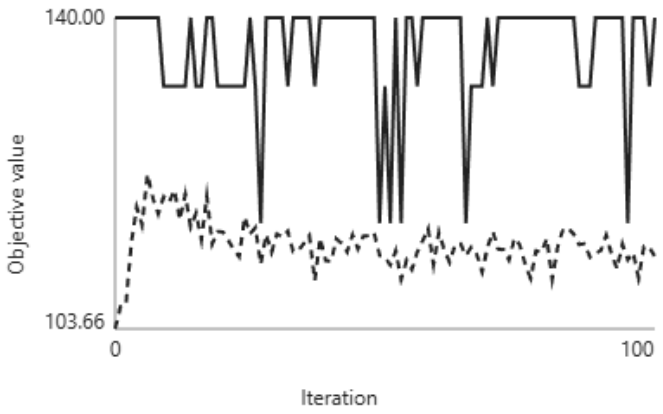
	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot** 

Variasi 3: Populasi = 70

Percobaan ke-1	Ringkasan Hasil	Objective: Shipping fee Trial: 1 of 1 Final objective: 140 Mean objective: 140 Vehicles loaded: 4 of 5 Trial runtime: 7801.73 ms Total runtime: 7801.81 ms Iterations: 100 Population: 70

	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot** 
Percobaan ke-2	Ringkasan Hasil	Objective: Shipping fee Trial: 1 of 1 Final objective: 140 Mean objective: 140 Vehicles loaded: 4 of 5 Trial runtime: 9342.59 ms Total runtime: 9342.69 ms Iterations: 100 Population: 70

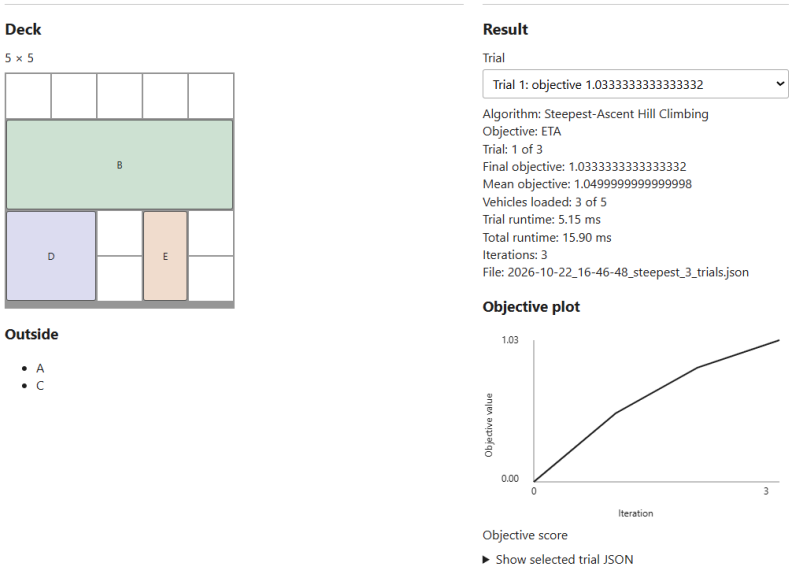
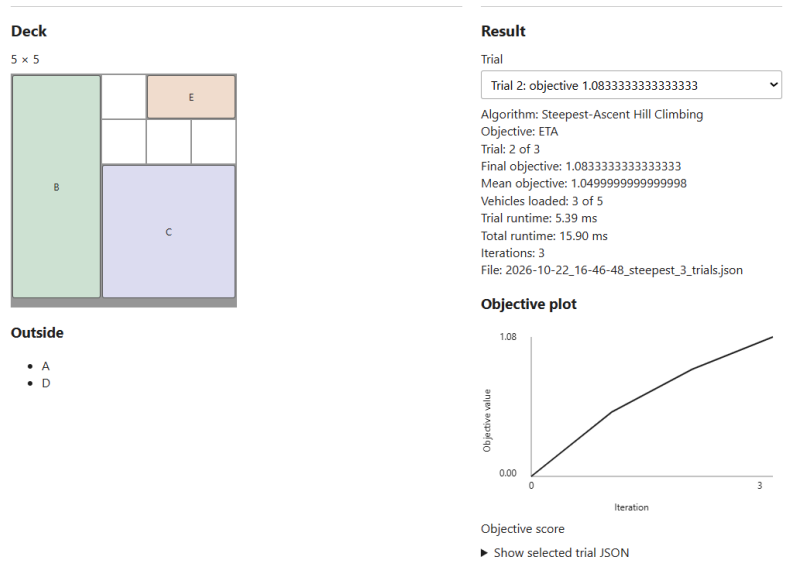
	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot**  Solid = max; dashed = average
Percobaan ke-3	Ringkasan Hasil	Objective: Shipping fee Trial: 1 of 1 Final objective: 140 Mean objective: 140 Vehicles loaded: 4 of 5 Trial runtime: 7587.19 ms Total runtime: 7587.28 ms Iterations: 100 Population: 70

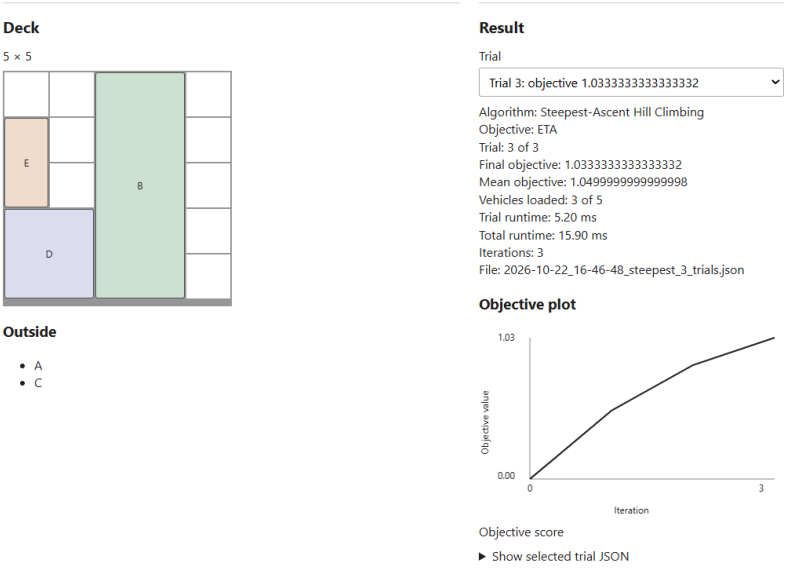
	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot**  Solid = max; dashed = average

Hasil Eksperimen (Nilai Objective: ETA)

Steepest Ascent Hill Climbing

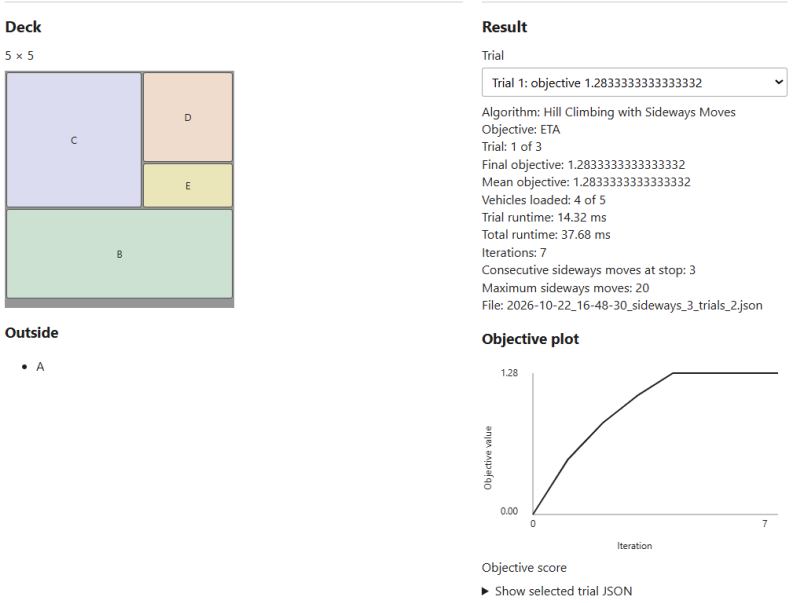
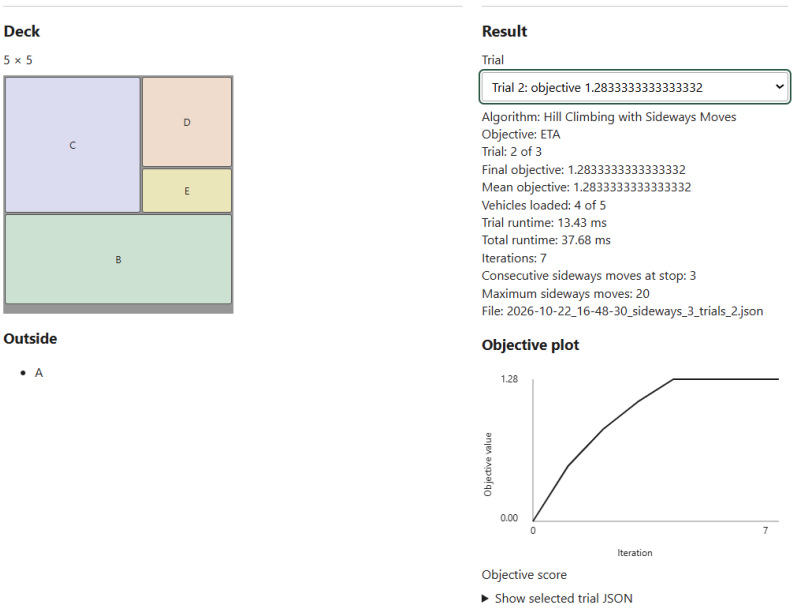
Percobaan ke-1	Ringkasan Hasil	Objective: ETA Trial: 1 of 3 Final objective: 1.0333333333333332 Mean objective: 1.0499999999999998 Vehicles loaded: 3 of 5

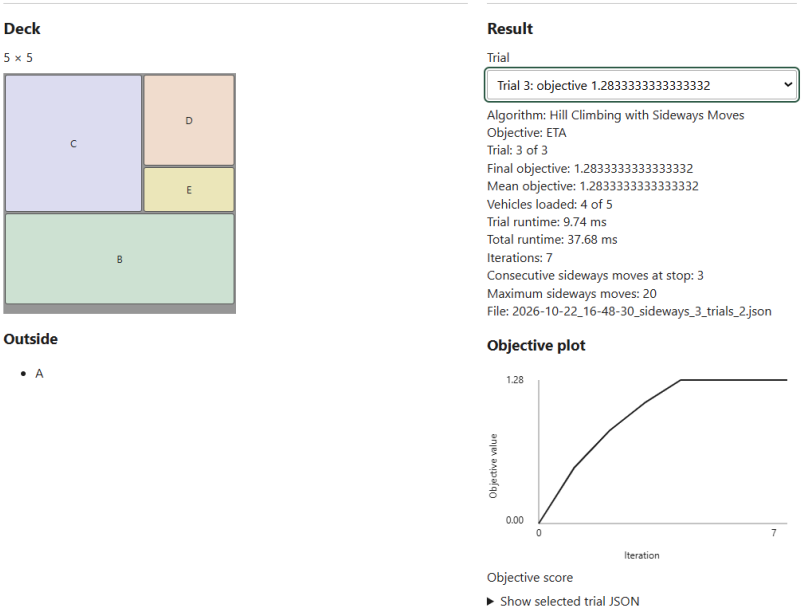
		Trial runtime: 5.15 ms Total runtime: 15.90 ms Iterations: 3
	Visualisasi	
Percobaan ke-2	Ringkasan Hasil	Objective: ETA Trial: 2 of 3 Final objective: 1.0833333333333333 Mean objective: 1.0499999999999998 Vehicles loaded: 3 of 5 Trial runtime: 5.39 ms Total runtime: 15.90 ms
	Visualisasi	
Percobaan ke-3	Ringkasan Hasil	Objective: ETA Trial: 3 of 3 Final objective: 1.0333333333333332

		Mean objective: 1.0499999999999998 Vehicles loaded: 3 of 5 Trial runtime: 5.20 ms Total runtime: 15.90 ms Iterations: 3
	Visualisasi	

Sideways Move Hill Climbing

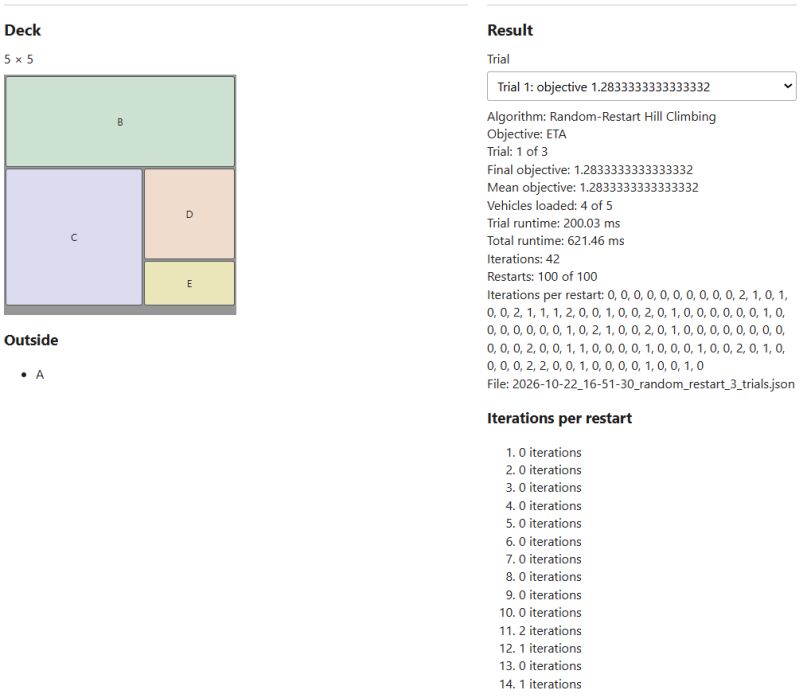
Percobaan ke-1	Ringkasan Hasil	Objective: ETA Trial: 1 of 3 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 14.32 ms Total runtime: 37.68 ms Iterations: 7 Consecutive sideways moves at stop: 3 Maximum sideways moves: 20

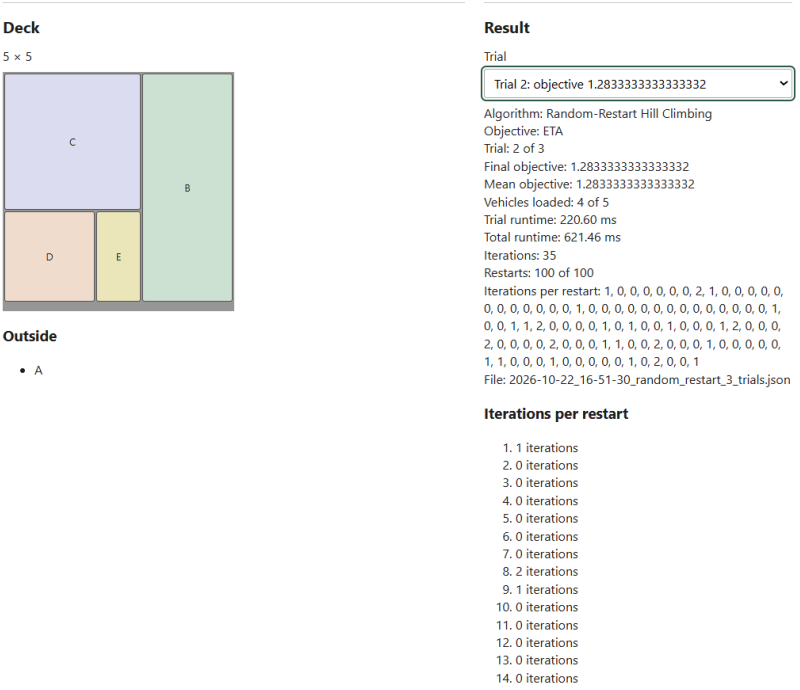
	Visualisasi	
Percobaan ke-2	Ringkasan Hasil	Objective: ETA Trial: 2 of 3 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 13.43 ms Total runtime: 37.68 ms Iterations: 7 Consecutive sideways moves at stop: 3 Maximum sideways moves: 20
	Visualisasi	
Percobaan ke-3	Ringkasan Hasil	Objective: ETA Trial: 3 of 3

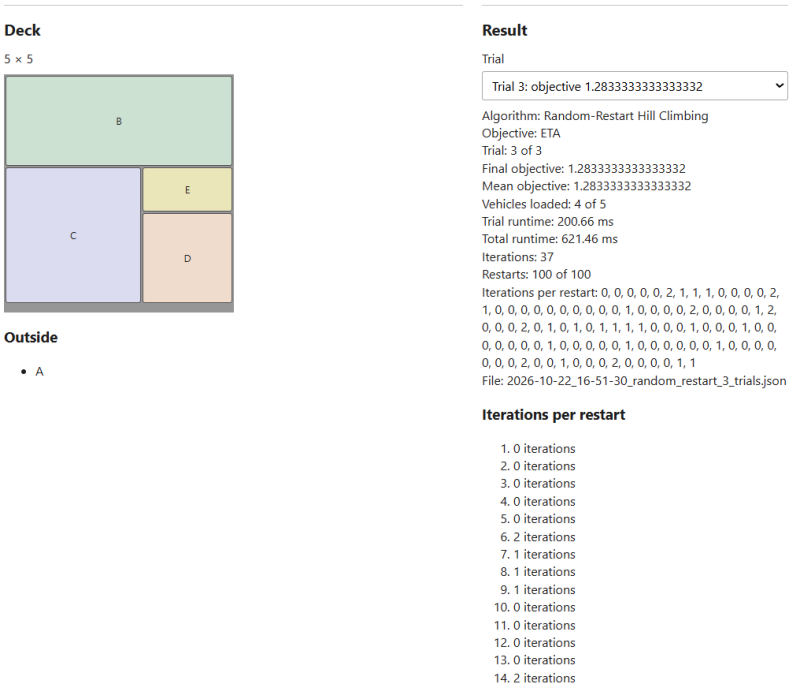
		Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 9.74 ms Total runtime: 37.68 ms Iterations: 7 Consecutive sideways moves at stop: 3 Maximum sideways moves: 20
	Visualisasi	

Random Restart Hill Climbing

Percobaan ke-1	Ringkasan Hasil	Objective: ETA Trial: 1 of 3 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 200.03 ms Total runtime: 621.46 ms Iterations: 42 Restarts: 100 of 100 Iterations per restart: 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 1, 0, 1, 0, 0, 2, 1, 1, 1, 2, 0, 0, 1, 0, 0, 2, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 2, 1, 0, 0, 2, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 1, 1, 0, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 2, 0, 1, 0, 0, 0, 0, 2, 2, 0, 0, 1, 0, 0, 0, 0, 1, 0, 0, 1, 0

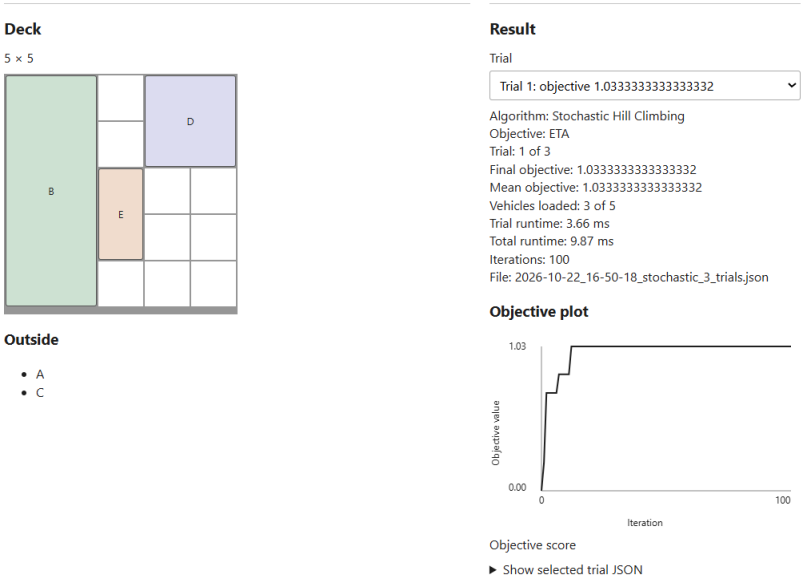
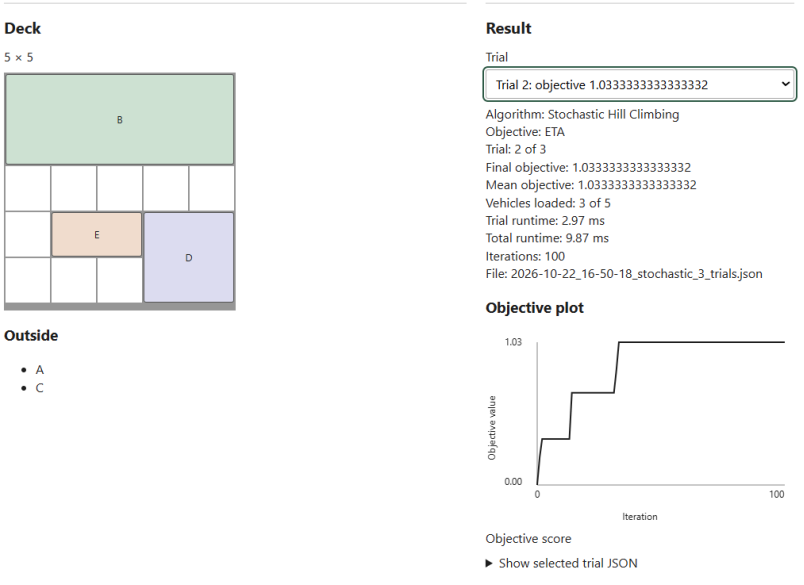
	Visualisasi	
Percobaan ke-2	Ringkasan Hasil	Objective: ETA Trial: 2 of 3 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 220.60 ms Total runtime: 621.46 ms Iterations: 35 Restarts: 100 of 100 Iterations per restart: 1, 0, 0, 0, 0, 0, 0, 2, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 1, 1, 2, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 0, 0, 0, 1, 2, 0, 0, 0, 2, 0, 0, 0, 0, 2, 0, 0, 0, 1, 1, 0, 0, 2, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 2, 0, 0, 1

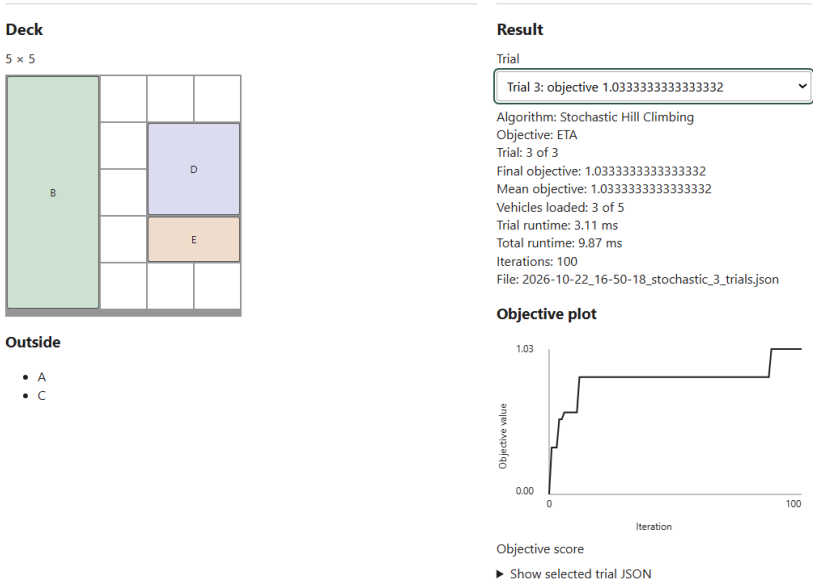
	Visualisasi	
Percobaan ke-3	Ringkasan Hasil	Objective: ETA Trial: 3 of 3 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 200.66 ms Total runtime: 621.46 ms Iterations: 37 Restarts: 100 of 100 Iterations per restart: 0, 0, 0, 0, 0, 2, 1, 1, 1, 0, 0, 0, 0, 2, 1, 0, 0, 0, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 2, 0, 0, 0, 0, 1, 2, 0, 0, 0, 2, 0, 1, 0, 1, 0, 1, 1, 1, 1, 0, 0, 0, 1, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 0, 0, 0, 0, 0, 0, 2, 0, 0, 1, 0, 0, 0, 2, 0, 0, 0, 0, 1, 1

	Visualisasi	

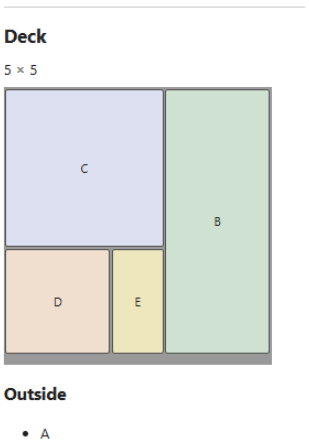
Stochastic Hill Climbing

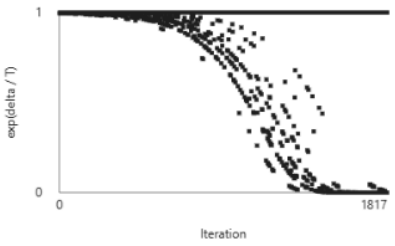
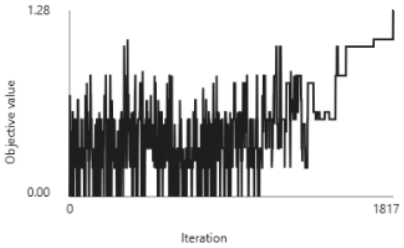
Percobaan ke-1	Ringkasan Hasil	Objective: ETA Trial: 1 of 3 Final objective: 1.0333333333333332 Mean objective: 1.0333333333333332 Vehicles loaded: 3 of 5 Trial runtime: 3.66 ms Total runtime: 9.87 ms Iterations: 100

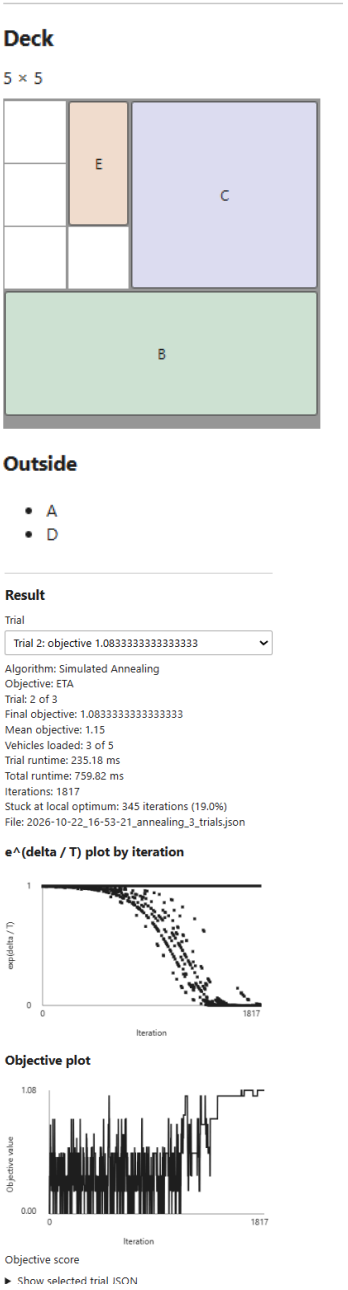
	Visualisasi	
Percobaan ke-2	Ringkasan Hasil	Objective: ETA Trial: 2 of 3 Final objective: 1.0333333333333332 Mean objective: 1.0333333333333332 Vehicles loaded: 3 of 5 Trial runtime: 2.97 ms Total runtime: 9.87 ms Iterations: 100
	Visualisasi	
Percobaan ke-3	Ringkasan Hasil	Objective: ETA Trial: 3 of 3 Final objective: 1.0333333333333332 Mean objective: 1.0333333333333332 Vehicles loaded: 3 of 5

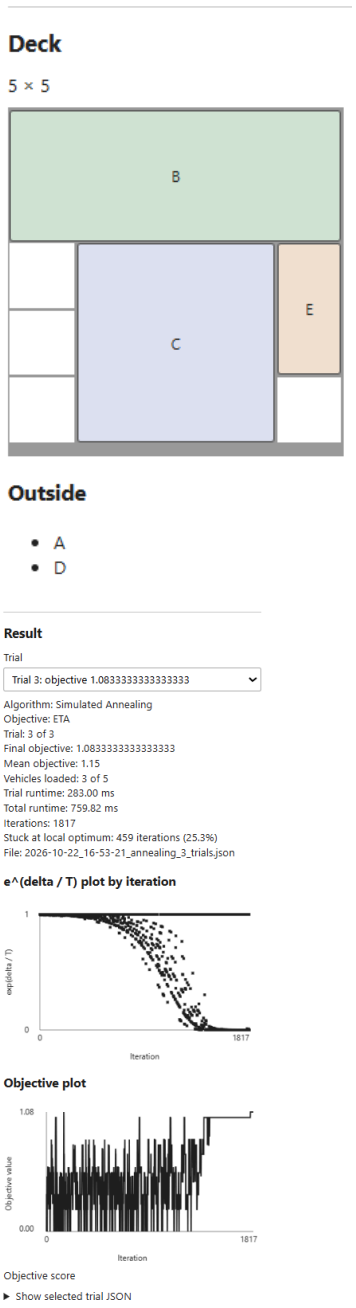
		Trial runtime: 3.11 ms Total runtime: 9.87 ms Iterations: 100
	Visualisasi	

Simulated Annealing

Percobaan ke-1	Ringkasan Hasil	Objective: ETA Trial: 1 of 3 Final objective: 1.2833333333333332 Mean objective: 1.15 Vehicles loaded: 4 of 5 Trial runtime: 241.46 ms Total runtime: 759.82 ms Iterations: 1817 Stuck at local optimum: 260 iterations (14.3%)
	Visualisasi	

		Result Trial Trial 1: objective 1.2833333333333332 Algorithm: Simulated Annealing Objective: ETA Trial: 1 of 3 Final objective: 1.2833333333333332 Mean objective: 1.15 Vehicles loaded: 4 of 5 Trial runtime: 241.46 ms Total runtime: 759.82 ms Iterations: 1817 Stuck at local optimum: 260 iterations (14.3%) File: 2026-10-22_16-53-21_annealing_3_trials.json **e^(delta / T) plot by iteration**  **Objective plot**  Objective score ▶ Show selected trial JSON
Percobaan ke-2	Ringkasan Hasil	Objective: ETA Trial: 2 of 3 Final objective: 1.0833333333333333 Mean objective: 1.15 Vehicles loaded: 3 of 5 Trial runtime: 235.18 ms Total runtime: 759.82 ms Iterations: 1817 Stuck at local optimum: 345 iterations (19.0%)

	Visualisasi	
Percobaan ke-3	Ringkasan Hasil	Objective: ETA Trial: 3 of 3 Final objective: 1.0833333333333333 Mean objective: 1.15 Vehicles loaded: 3 of 5 Trial runtime: 283.00 ms Total runtime: 759.82 ms Iterations: 1817 Stuck at local optimum: 459 iterations (25.3%)

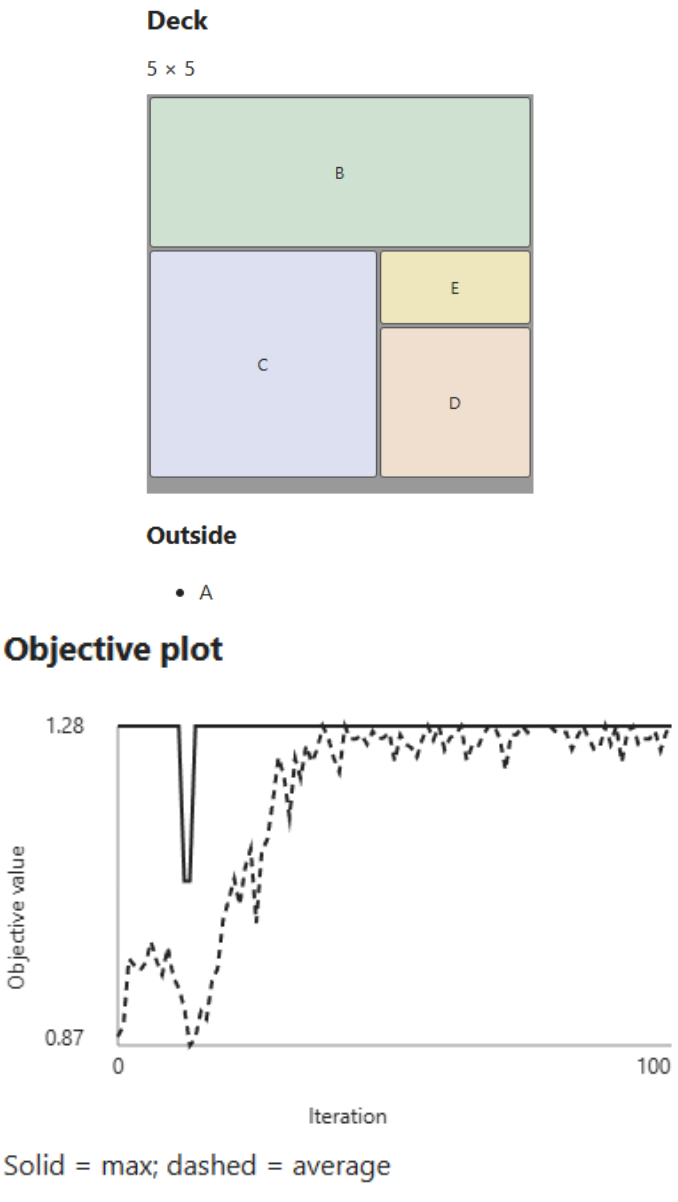
	Visualisasi	

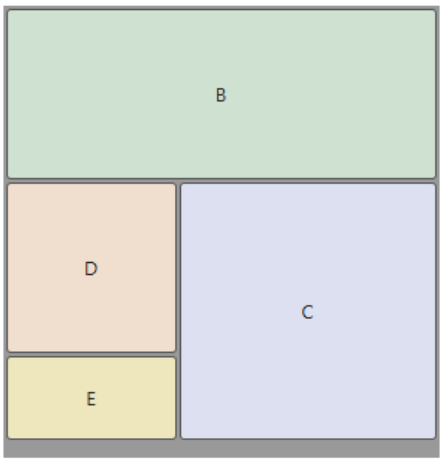
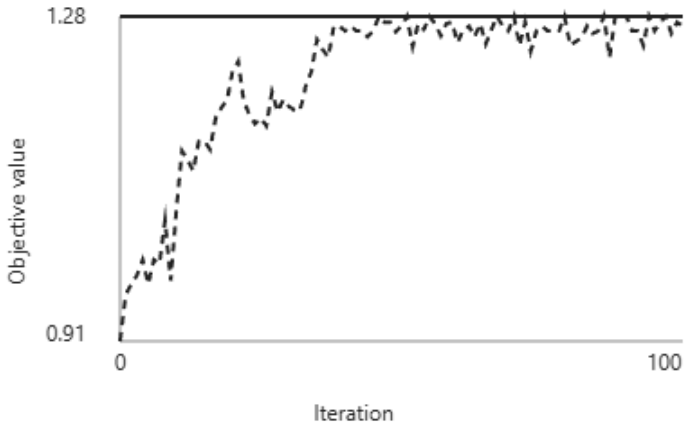
Genetic Algorithm

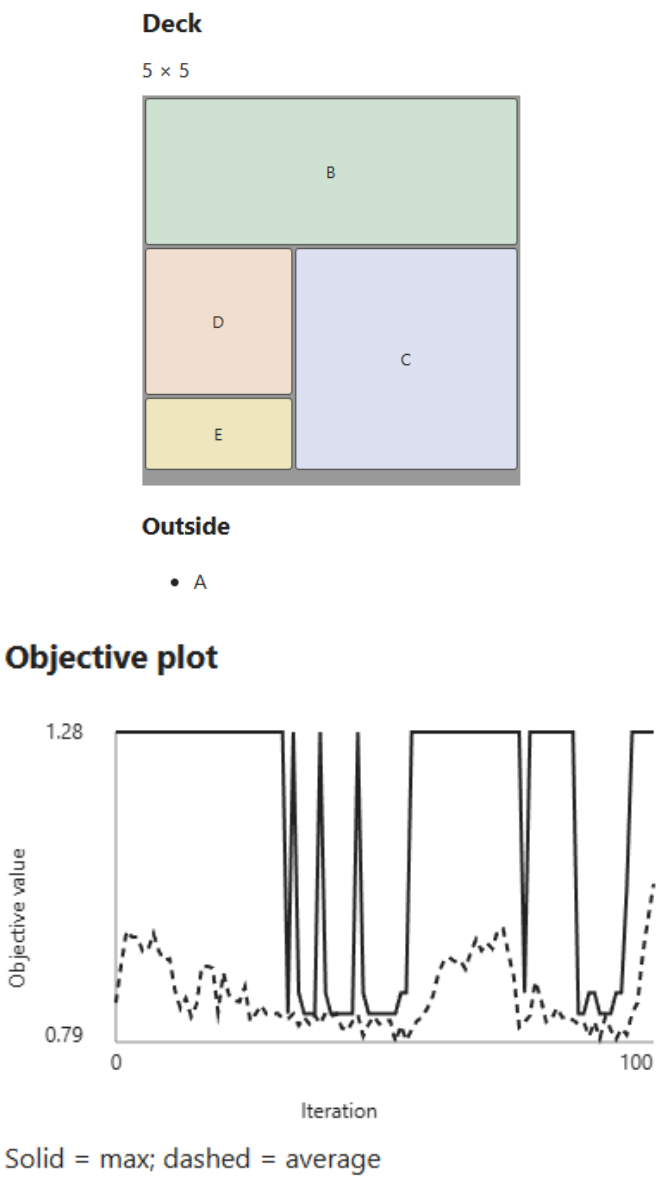
Pada plot di hasil Genetic Algorithm ini, garis tebal (solid) menandakan Best Fitness selama trial, dan garis putus-putus (dashed) menandakan Average Fitness per Iterasi.

Jumlah Populasi Sebagai Kontrol (Populasi = 30)

Variasi 1: Iterasi = 100

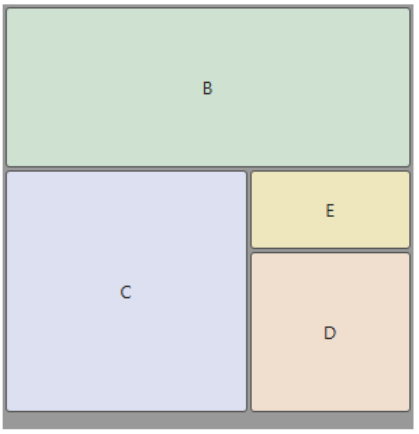
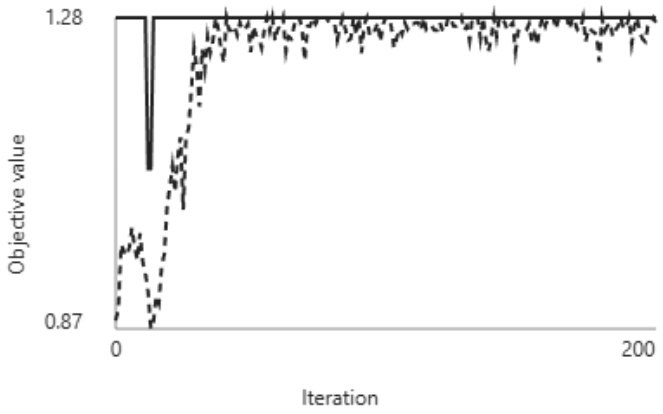
Percobaan ke-1	Ringkasan Hasil	Objective: ETA Trial: 1 of 1 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 1464.58 ms Total runtime: 1464.65 ms Iterations: 100 Population: 30
	Visualisasi	
Percobaan ke-2	Ringkasan Hasil	Objective: ETA Trial: 1 of 1 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5

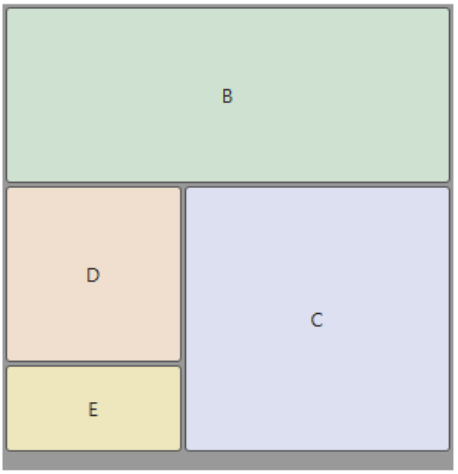
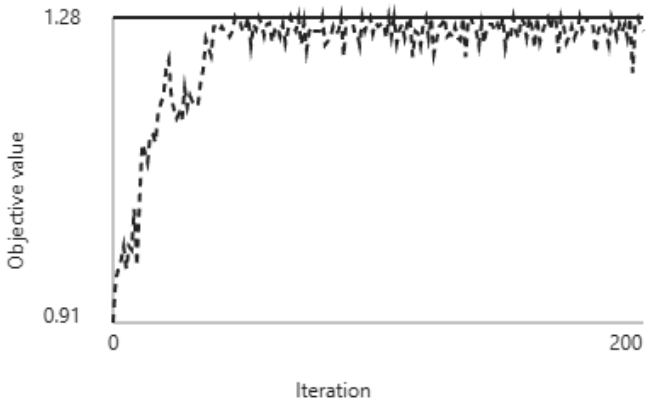
		Trial runtime: 1283.65 ms Total runtime: 1283.75 ms Iterations: 100 Population: 30
	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot**  Solid = max; dashed = average
Percobaan ke-3	Ringkasan Hasil	Objective: ETA Trial: 1 of 1 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 3944.26 ms Total runtime: 3944.36 ms

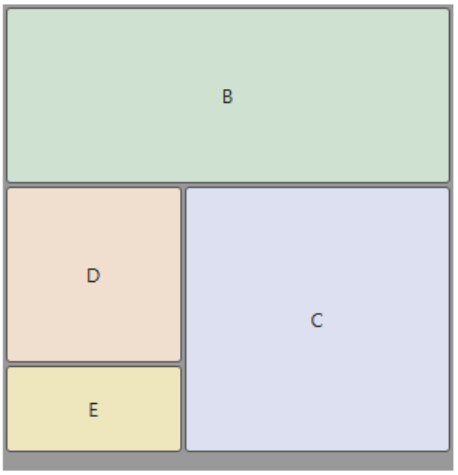
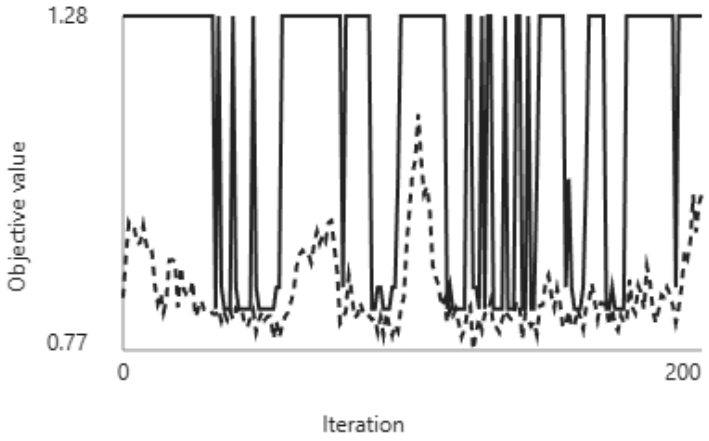
		Iterations: 100 Population: 30
	Visualisasi	

Variasi 2: Iterasi = 200

Percobaan ke-1	Ringkasan Hasil	Objective: ETA Trial: 1 of 1 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 4529.63 ms Total runtime: 4529.74 ms Iterations: 200 Population: 30

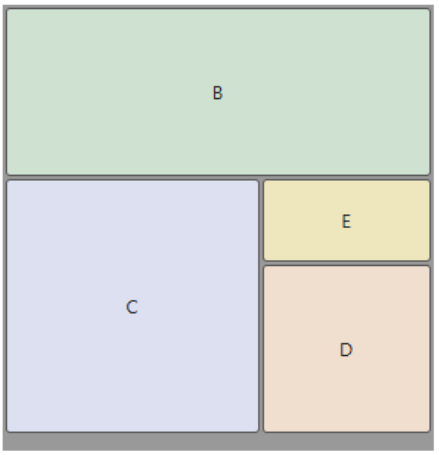
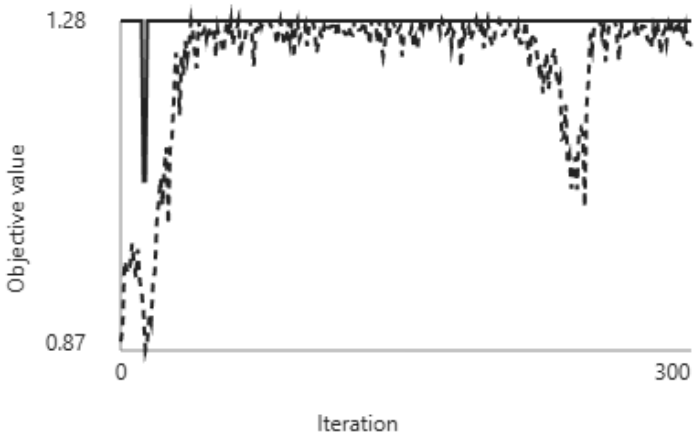
	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot**  Solid = max; dashed = average
Percobaan ke-2	Ringkasan Hasil	Objective: ETA Trial: 1 of 1 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 1920.55 ms Total runtime: 1920.61 ms Iterations: 200 Population: 30

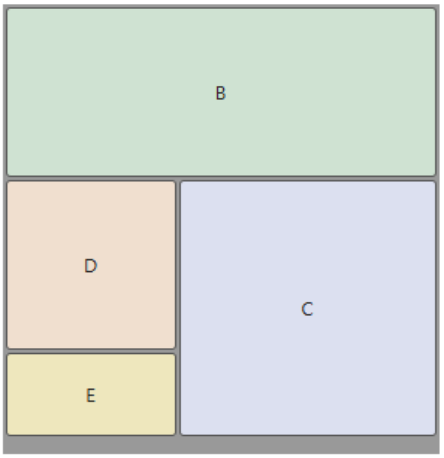
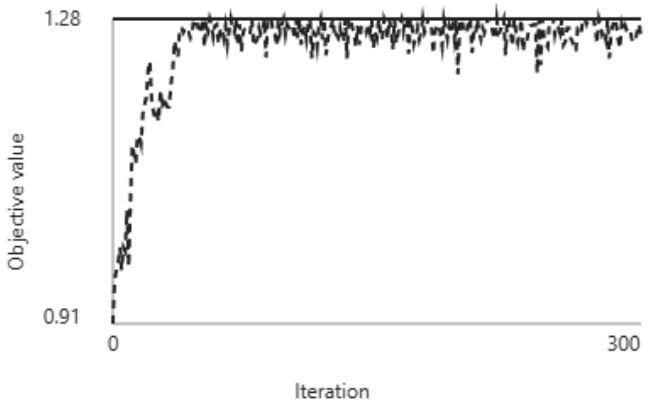
	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot**  Solid = max; dashed = average
Percobaan ke-3	Ringkasan Hasil	Objective: ETA Trial: 1 of 1 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 7542.72 ms Total runtime: 7542.79 ms Iterations: 200 Population: 30

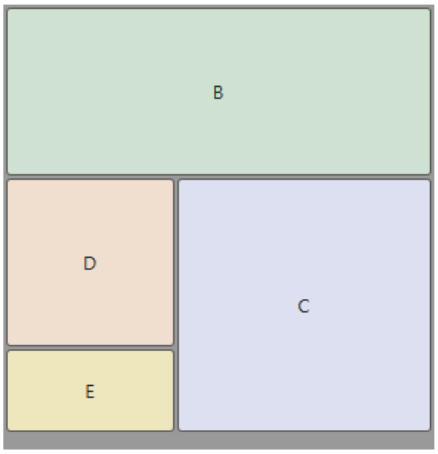
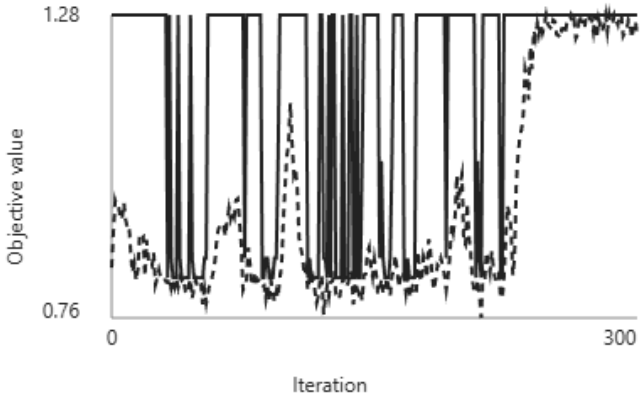
	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot**  Solid = max; dashed = average

Variasi 3: Iterasi = 300

Percobaan ke-1	Ringkasan Hasil	Objective: ETA Trial: 1 of 1 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 4129.00 ms Total runtime: 4129.09 ms Iterations: 300

		Population: 30
	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot**  Solid = max; dashed = average
Percobaan ke-2	Ringkasan Hasil	Objective: ETA Trial: 1 of 1 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 3244.20 ms Total runtime: 3244.29 ms Iterations: 300 Population: 30

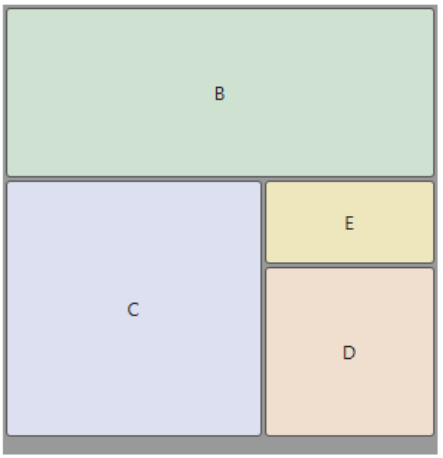
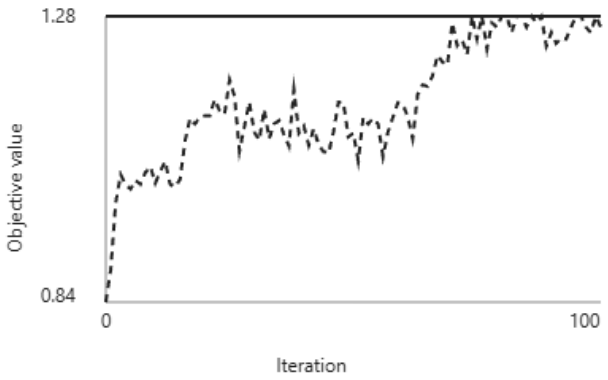
	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot**  Solid = max; dashed = average
Percobaan ke-3	Ringkasan Hasil	Objective: ETA Trial: 1 of 1 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 7670.19 ms Total runtime: 7670.29 ms Iterations: 300 Population: 30

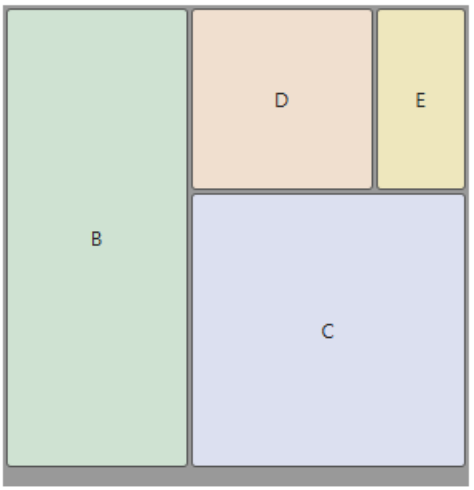
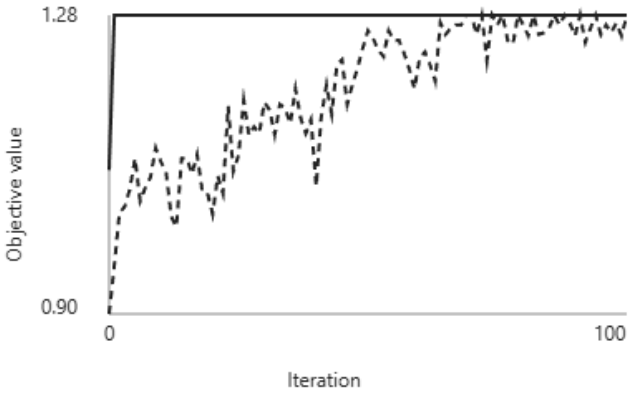
	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot**  Solid = max; dashed = average

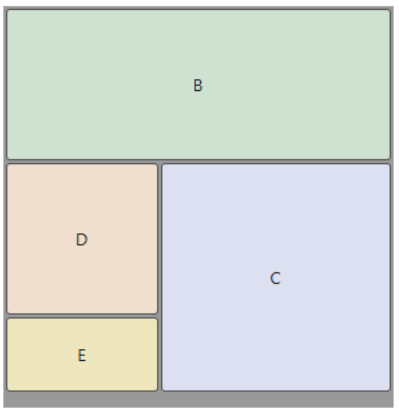
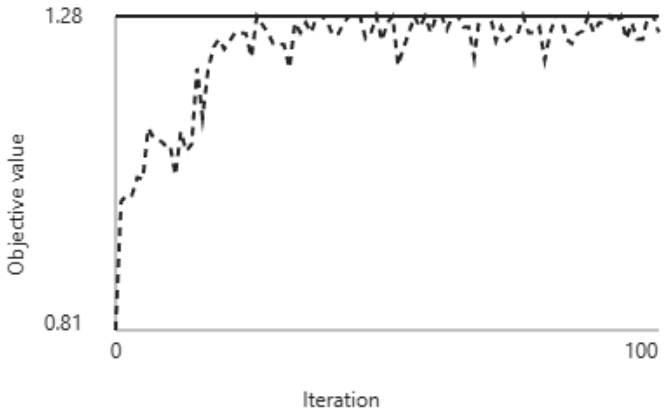
Jumlah Iterasi Sebagai Kontrol (Iterasi = 100)

Variasi 1: Populasi = 20

Percobaan ke-1	Ringkasan Hasil	Objective: ETA Trial: 1 of 1 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 1926.66 ms Total runtime: 1926.83 ms Iterations: 100 Population: 20

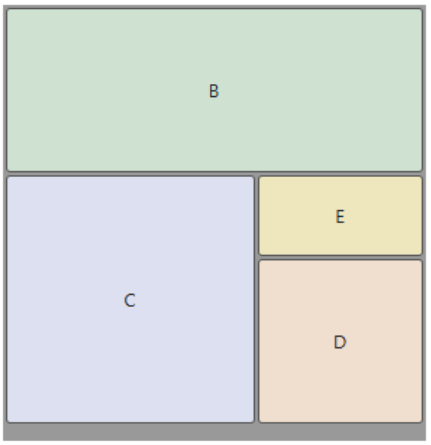
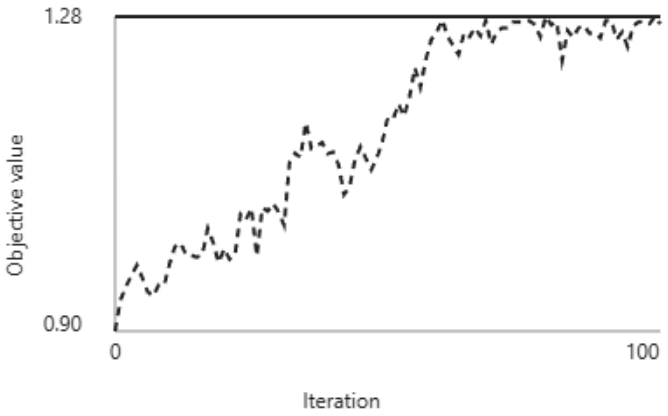
	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot**  Solid = max; dashed = average
Percobaan ke-2	Ringkasan Hasil	Objective: ETA Trial: 1 of 1 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 1272.34 ms Total runtime: 1272.45 ms Iterations: 100 Population: 20

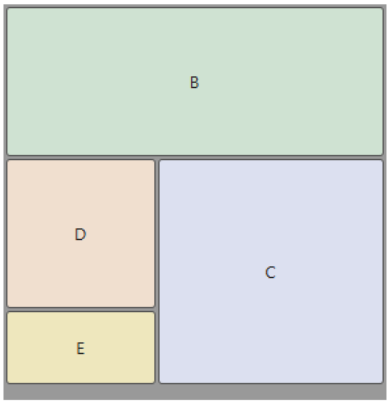
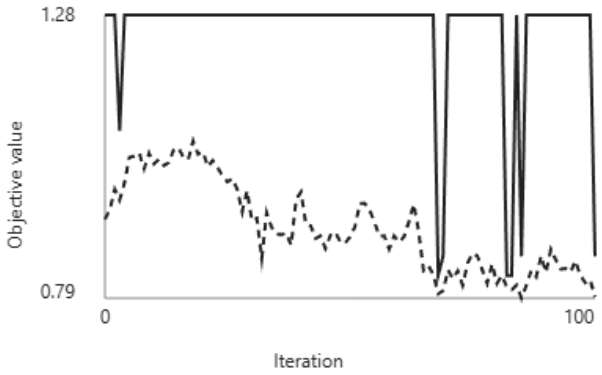
	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot**  Solid = max; dashed = average
Percobaan ke-3	Ringkasan Hasil	Objective: ETA Trial: 1 of 1 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 920.36 ms Total runtime: 920.44 ms Iterations: 100 Population: 20

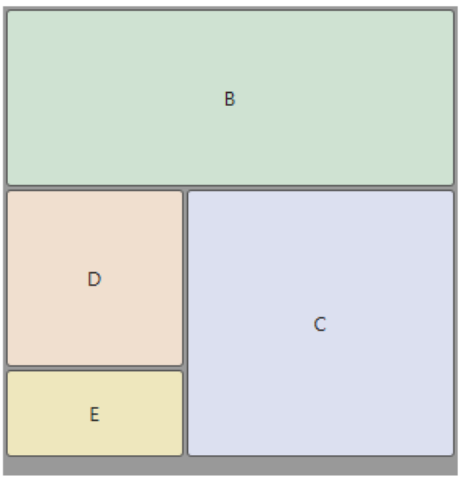
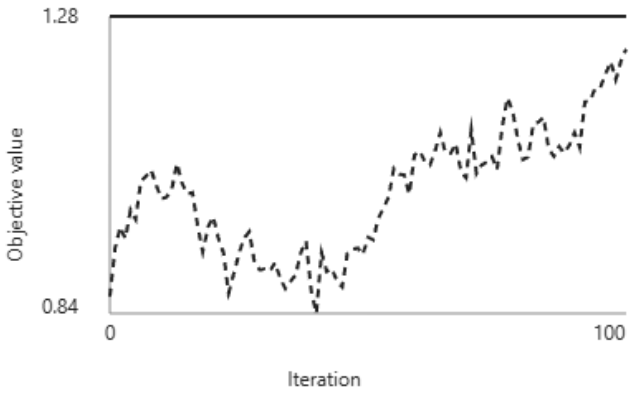
	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot**  Solid = max; dashed = average

Variasi 2: Populasi = 50

Percobaan ke-1	Ringkasan Hasil	Objective: ETA Trial: 1 of 1 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 3959.42 ms Total runtime: 3959.50 ms Iterations: 100 Population: 50

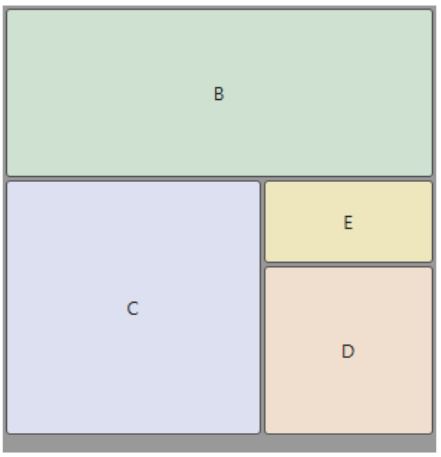
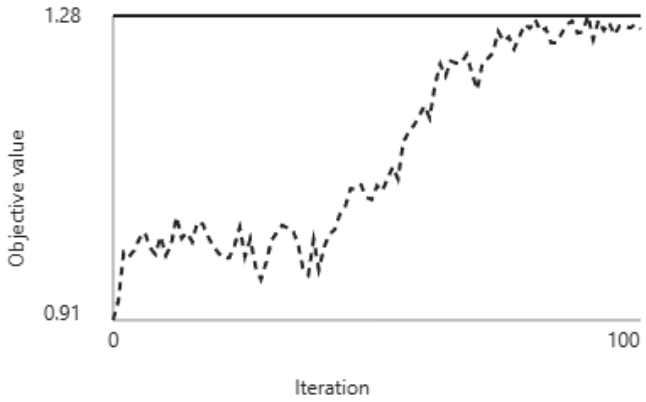
	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot**  Solid = max; dashed = average
Percobaan ke-2	Ringkasan Hasil	Objective: ETA Trial: 1 of 1 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 3852.66 ms Total runtime: 3852.75 ms Iterations: 100 Population: 50

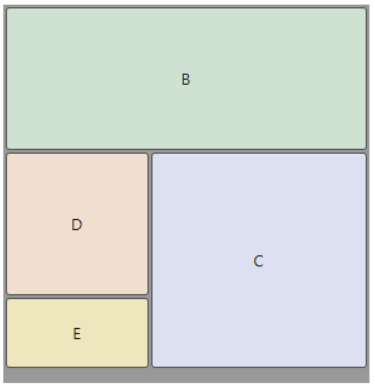
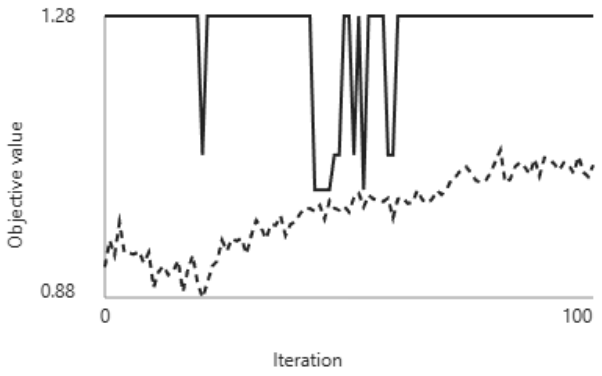
	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot**  Solid = max; dashed = average
Percobaan ke-3	Ringkasan Hasil	Objective: ETA Trial: 1 of 1 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 6369.69 ms Total runtime: 6369.92 ms Iterations: 100 Population: 50

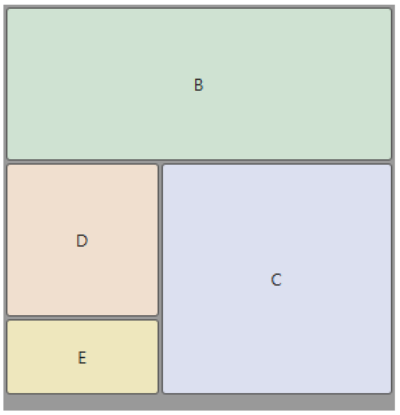
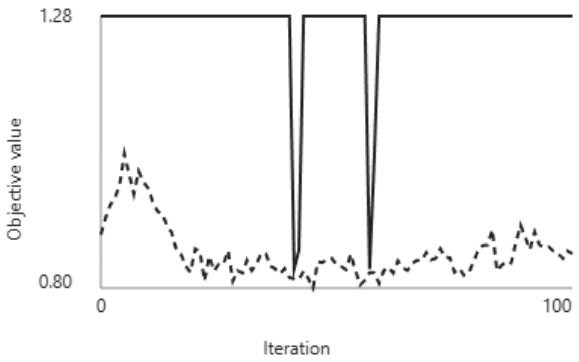
	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot**  Solid = max; dashed = average

Variasi 3: Populasi = 70

Percobaan ke-1	Ringkasan Hasil	Objective: ETA Trial: 1 of 1 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 4593.64 ms Total runtime: 4593.82 ms Iterations: 100 Population: 70

	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot**  Solid = max; dashed = average
Percobaan ke-2	Ringkasan Hasil	Objective: ETA Trial: 1 of 1 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 3792.55 ms Total runtime: 3792.64 ms Iterations: 100 Population: 70

	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot**  Solid = max; dashed = average
Percobaan ke-3	Ringkasan Hasil	Objective: ETA Trial: 1 of 1 Final objective: 1.2833333333333332 Mean objective: 1.2833333333333332 Vehicles loaded: 4 of 5 Trial runtime: 7500.45 ms Total runtime: 7500.53 ms Iterations: 100 Population: 70

	Visualisasi	**Deck** 5 × 5  **Outside** • A **Objective plot**  Solid = max; dashed = average

Analisis Hasil

Algoritma	Rata-Rata Nilai Objective Shipping Fee	Rata-Rata Nilai Objective ETA	Rata-Rata Durasi
Steepest Ascent Hill Climbing	105.33	1.0500	0.00344
Sideways Move Hill Climbing	116.00	1.2833	0.00715

Random Restart Hill Climbing	140.00	1.2833	0.14361
Stochastic Hill Climbing	126.00	1.0333	0.00238
Simulated Annealing	134.67	1.1500	0.16249
Genetic Algorithm	139.56	1.2833	5.76421

Pertanyaan Analisis

1. **Seberapa dekat tiap-tiap algoritma bisa mendekati global optima dan mengapa hasilnya demikian?**

   Berdasarkan hasil eksperimen, Random Restart Hill Climbing berhasil mencapai *global optimum* secara konsisten pada kedua *objective function*, yaitu 140 untuk *Shipping Fee* dan 1,2833 untuk ETA. *Genetic Algorithm* juga menghasilkan nilai yang mendekati optimal, dengan rata-rata *Shipping Fee* sebesar 139,56 atau 99,68% dari *global optimum*, serta ETA sebesar 1,2833.

   Random Restart dapat mencapai hasil optimal karena melakukan pencarian berulang dari berbagai *initial state* sehingga memiliki peluang lebih besar untuk menemukan solusi terbaik. Sementara itu, *Genetic Algorithm* menggunakan populasi, seleksi, *crossover*, dan mutasi untuk menghasilkan solusi baru. Dari 18 percobaan *Shipping Fee*, terdapat satu percobaan yang menghasilkan nilai akhir 132, sedangkan percobaan lainnya mencapai 140. Hal ini dapat terjadi karena algoritma mengembalikan individu terbaik dari generasi terakhir, sehingga solusi yang lebih baik dari generasi sebelumnya dapat hilang.

   Pada objective Shipping Fee, Simulated Annealing menghasilkan rata-rata 134,67 atau 96,19% dari global optimum, diikuti Stochastic Hill Climbing sebesar 90%, Sideways Move sebesar 82,86%, dan Steepest Ascent sebesar 75,24%.

   Steepest Ascent memiliki hasil paling rendah karena pencarian berhenti ketika tidak ditemukan neighbor dengan nilai lebih tinggi, sehingga berpotensi terjebak pada local optimum. Sideways Move dapat melewati kondisi ketika beberapa neighbor memiliki nilai yang sama. Stochastic Hill Climbing melakukan pencarian melalui neighbor acak. Simulated Annealing memperoleh hasil yang lebih mendekati optimum karena masih memungkinkan perpindahan ke state dengan nilai lebih rendah berdasarkan probabilitas tertentu.

Pada objective ETA, Sideways Move, Random Restart, dan Genetic Algorithm mencapai 100% dari global optimum. Sementara itu, Simulated Annealing mencapai rata-rata 89,61%, Steepest Ascent 81,82%, dan Stochastic Hill Climbing 80,52%.

Hasil tersebut menunjukkan bahwa kemampuan algoritma mendekati global optimum dipengaruhi oleh mekanisme pencarian dan objective function yang digunakan. Algoritma dengan mekanisme eksplorasi lebih luas cenderung memiliki peluang lebih besar untuk menemukan solusi optimal, meskipun hasilnya tetap bergantung pada kondisi awal dan proses pencarian.

2. Bagaimana perbandingan hasil pencarian tiap-tiap algoritma dengan algoritma local search yang lain?

Berdasarkan hasil eksperimen pada *objective Shipping Fee*, Random Restart Hill Climbing menghasilkan rata-rata tertinggi sebesar 140, diikuti *Genetic Algorithm* sebesar 139,56, Simulated Annealing sebesar 134,67, Stochastic Hill Climbing sebesar 126, Sideways Move Hill Climbing sebesar 116, dan Steepest Ascent Hill Climbing sebesar 105,33.

Perbedaan ini dipengaruhi oleh mekanisme pencarian masing-masing algoritma. Steepest Ascent hanya memilih neighbor dengan nilai lebih tinggi sehingga mudah berhenti pada local optimum. Sideways Move memberikan kesempatan untuk berpindah ke neighbor dengan nilai yang sama, tetapi peningkatannya sedikit. Stochastic Hill Climbing menghasilkan nilai lebih tinggi melalui pemilihan neighbor secara acak. Simulated Annealing dapat menerima perpindahan ke state yang lebih buruk untuk memperluas eksplorasi. Random Restart memperoleh hasil optimal dengan menjalankan pencarian dari berbagai initial state. Genetic Algorithm menggunakan populasi, seleksi, crossover, dan mutasi untuk menghasilkan kandidat solusi.

Pada objective ETA, terdapat perbedaan hasil. Sideways Move Hill Climbing mampu mencapai nilai optimal sebesar 1,2833, sama seperti Random Restart dan Genetic Algorithm. Sementara itu, Simulated Annealing menghasilkan rata-rata 1,15, Steepest Ascent sebesar 1,05, dan Stochastic Hill Climbing sebesar 1,0333.

Hal ini menunjukkan bahwa efektivitas suatu algoritma dapat berbeda berdasarkan *objective function* yang digunakan. Secara keseluruhan, Random Restart Hill Climbing memberikan hasil paling konsisten dengan mencapai *global optimum* pada seluruh percobaan. *Genetic Algorithm* juga menghasilkan nilai yang mendekati optimal, meskipun terdapat satu percobaan dengan hasil lebih rendah. Namun, hasil tersebut dipengaruhi oleh karakteristik persoalan dan *initial state* yang digunakan, sehingga belum dapat disimpulkan bahwa kedua algoritma selalu memberikan hasil terbaik pada setiap permasalahan.

3. **Bagaimana perbandingan durasi proses pencarian tiap algoritma relatif terhadap algoritma lainnya?**

   Berdasarkan hasil eksperimen, Stochastic Hill Climbing memiliki rata-rata durasi pencarian paling singkat sebesar 0,00238 detik, diikuti Steepest Ascent Hill Climbing sebesar 0,00344 detik dan Sideways Move Hill Climbing sebesar 0,00715 detik. Ketiga algoritma ini memiliki durasi pencarian yang relatif singkat karena proses pencariannya lebih sederhana. Stochastic Hill Climbing hanya mengevaluasi satu neighbor acak pada setiap iterasi. Steepest Ascent dan Sideways Move melakukan evaluasi terhadap seluruh neighbor yang tersedia, sehingga membutuhkan waktu lebih lama.

   Random Restart Hill Climbing memiliki rata-rata durasi sebesar 0,14361 detik dan Simulated Annealing sebesar 0,16249 detik. Random Restart membutuhkan waktu lebih lama karena menjalankan Steepest Ascent berulang kali dengan initial state yang berbeda sebanyak 100 restart. Simulated Annealing melakukan pencarian hingga 1817 iterasi dan pemeriksaan kondisi local optimum, sehingga proses evaluasinya juga membutuhkan waktu lebih banyak.

   Genetic Algorithm memiliki rata-rata durasi paling lama sebesar 5,76421 detik. Hal ini disebabkan oleh proses pembentukan populasi baru melalui seleksi, crossover, mutasi, dan repair pada setiap generasi. Fungsi repair juga melakukan pemeriksaan berbagai kemungkinan posisi kendaraan untuk menghasilkan state yang valid. Percobaan Genetic Algorithm menggunakan variasi jumlah populasi dan iterasi yang berbeda, sehingga rata-rata durasinya mencakup beban komputasi yang lebih besar.

   Secara keseluruhan, algoritma dengan durasi lebih lama cenderung melakukan proses pencarian yang lebih banyak. Namun, waktu pencarian yang lebih lama tidak selalu menghasilkan nilai *objective* yang lebih tinggi. Random Restart Hill Climbing berhasil mencapai *global optimum* secara konsisten dengan durasi yang jauh lebih singkat dibandingkan *Genetic Algorithm*. Meskipun *Genetic Algorithm* membutuhkan waktu lebih lama, hasilnya masih mendekati *global optimum* pada seluruh percobaan.

4. **Seberapa konsisten hasil akhir yang didapatkan dari tiap-tiap eksperimen yang dilakukan?**

   Berdasarkan hasil eksperimen pada *objective Shipping Fee*, Sideways Move Hill Climbing dan Random Restart Hill Climbing menunjukkan hasil yang konsisten dengan nilai masing-masing sebesar 116 dan 140 pada seluruh percobaan. *Genetic Algorithm* juga memberikan hasil yang cukup konsisten, dengan 17 dari 18 percobaan mencapai nilai 140 dan satu percobaan menghasilkan nilai 132. Steepest Ascent Hill Climbing menghasilkan nilai 100 pada dua percobaan dan 116 pada satu

percobaan. Stochastic Hill Climbing memperoleh nilai 114 dan 132, sementara Simulated Annealing menghasilkan nilai 132 dan 140. Perbedaan hasil ini menunjukkan bahwa beberapa algoritma masih dipengaruhi oleh *initial state* dan proses pencarian yang dilakukan.

Pada objective ETA, Sideways Move Hill Climbing, Random Restart Hill Climbing, dan Genetic Algorithm kembali menunjukkan hasil yang konsisten dengan nilai objective sebesar 1,2833. Stochastic Hill Climbing juga konsisten dengan nilai 1,0333 pada seluruh percobaan, meskipun belum mencapai global optimum. Steepest Ascent menghasilkan nilai antara 1,0333 hingga 1,0833, dan Simulated Annealing menghasilkan nilai antara 1,0833 hingga 1,2833.

Secara keseluruhan, Random Restart Hill Climbing menunjukkan konsistensi terbaik karena selalu mencapai *global optimum* pada kedua *objective function*. *Genetic Algorithm* juga menunjukkan hasil yang cukup konsisten, terutama pada ETA yang selalu mencapai nilai optimal. Namun, pada *Shipping Fee*, terdapat satu percobaan yang menghasilkan nilai lebih rendah. Hal ini dapat terjadi karena populasi diganti pada setiap generasi dan algoritma mengambil individu terbaik dari generasi terakhir, sehingga nilai *fitness* akhir dapat lebih rendah dibandingkan generasi sebelumnya.

5. Bagaimana pengaruh banyak iterasi dan jumlah populasi terhadap hasil akhir pencarian pada Genetic Algorithm?

Berdasarkan hasil eksperimen, penambahan jumlah iterasi pada *Genetic Algorithm* belum menunjukkan peningkatan nilai *objective* akhir yang konsisten. Dengan jumlah populasi tetap sebesar 30, percobaan menggunakan 100, 200, dan 300 iterasi sebagian besar berhasil mencapai *global optimum*. Namun, terdapat satu percobaan *Shipping Fee* dari seluruh konfigurasi yang menghasilkan nilai 132. Pada *objective ETA*, seluruh percobaan tetap mencapai nilai optimal sebesar 1,2833.

Penambahan jumlah iterasi juga meningkatkan durasi pencarian. Pada *objective Shipping Fee*, rata-rata durasi meningkat dari 3,75 detik pada 100 iterasi menjadi 9,76 detik pada 200 iterasi dan 14,70 detik pada 300 iterasi. Hal ini terjadi karena semakin banyak iterasi, semakin banyak pula proses seleksi, *crossover*, mutasi, dan *repair* yang dijalankan.

Pada variasi jumlah populasi dengan iterasi tetap sebesar 100, populasi 20, 50, dan 70 juga memberikan hasil yang sebagian besar mencapai nilai optimal. Perbedaan yang terlihat terdapat pada durasi pencarian. Pada *objective Shipping Fee*, rata-rata durasi meningkat dari 1,66 detik untuk populasi 20 menjadi 7,75 detik untuk populasi 50 dan 8,24 detik untuk populasi 70. Hal ini disebabkan oleh bertambahnya jumlah individu yang harus dievaluasi dan diproses pada setiap generasi.

Secara keseluruhan, penambahan jumlah iterasi dan populasi meningkatkan waktu pencarian tanpa memberikan peningkatan nilai *objective* yang berarti pada *test case* ini. Sebagian besar percobaan telah mencapai *global optimum*, tetapi terdapat satu percobaan dengan hasil lebih rendah. Hal ini menunjukkan bahwa jumlah iterasi dan populasi yang lebih besar belum tentu menghasilkan solusi akhir yang lebih baik, karena algoritma mengambil individu terbaik dari generasi terakhir. Pada permasalahan yang lebih kompleks, penambahan parameter tersebut dapat meningkatkan peluang menemukan solusi yang lebih baik.

Kesimpulan dan Saran

Kesimpulan

Berdasarkan hasil analisis dari seluruh eksperimen, setiap algoritma memiliki kemampuan yang berbeda dalam mencari solusi optimal. Random Restart Hill Climbing menunjukkan hasil terbaik karena berhasil mencapai *global optimum* secara konsisten pada kedua *objective function*, yaitu 140 untuk *Shipping Fee* dan 1,2833 untuk ETA. *Genetic Algorithm* juga menghasilkan nilai yang mendekati optimal, dengan satu percobaan *Shipping Fee* menghasilkan nilai 132 dan sisanya mencapai 140. Sementara itu, algoritma lainnya menghasilkan nilai yang bervariasi karena dipengaruhi oleh mekanisme pencarian, *initial state*, dan kemungkinan terjebak pada *local optimum*.

Dari segi durasi, Stochastic Hill Climbing memiliki waktu pencarian paling singkat, sedangkan Genetic Algorithm membutuhkan waktu paling lama karena melakukan proses seleksi, crossover, mutasi, dan repair. Random Restart Hill Climbing menjadi algoritma yang paling efisien dalam eksperimen ini karena mampu mencapai global optimum dengan durasi yang lebih singkat dibandingkan Genetic Algorithm.

Pada *Genetic Algorithm*, penambahan jumlah iterasi dan populasi belum memberikan peningkatan nilai *objective* akhir yang berarti karena sebagian besar percobaan sudah mencapai solusi optimal. Penambahan parameter tersebut juga meningkatkan durasi pencarian. Secara keseluruhan, hasil eksperimen menunjukkan bahwa pemilihan algoritma perlu mempertimbangkan kualitas solusi, konsistensi hasil, dan waktu pencarian berdasarkan karakteristik permasalahan yang diselesaikan.

Saran

Berdasarkan hasil eksperimen yang telah dilakukan, penggunaan algoritma Random Restart Hill Climbing lebih disarankan untuk permasalahan penempatan kendaraan pada kapal karena mampu mencapai global optimum secara konsisten dengan durasi pencarian yang relatif singkat. Sementara itu, penggunaan algoritma lain dapat disesuaikan dengan kebutuhan dan karakteristik permasalahan.

Untuk pengembangan selanjutnya, disarankan melakukan pengujian menggunakan test case yang lebih bervariasi, seperti penambahan jumlah kendaraan, ukuran deck, dan kapasitas kapal untuk mengetahui kemampuan setiap algoritma dalam menyelesaikan permasalahan yang lebih kompleks. Selain itu, pengujian parameter pada algoritma Simulated Annealing dan Genetic Algorithm dapat dilakukan untuk mencari konfigurasi yang menghasilkan solusi optimal dengan durasi lebih singkat. Jumlah percobaan juga dapat

ditambah agar hasil perbandingan kualitas solusi, konsistensi, dan durasi pencarian menjadi lebih representatif.

Referensi

GeeksforGeeks. . Genetic algorithms. https://www.geeksforgeeks.org/dsa/genetic-algorithms/

Slide Edunex IF3070 Dasar Inteligensi Artifisial 2026/2027, “Beyond Classical Search”

Pembagian Tugas

Nama	NIM	Pembagian Tugas
Nizham Rafa Lazuardi	18224049	Struktur data, steepest, genetic + laporan
Adrian Akhdan Assyauqi	18224055	Random restart, simulated annealing + laporan
Muhammad Azikra Wira Pratama	18224089	Sideways, stochastic, visualisasi + laporan

SURAT PERNYATAAN PENGGUNAAN AI

Dengan ini, ~~saya~~/kami*:

Nama/NIM : Nizham Rafa Lazuardi - 18224049

Adrian Akhdan Assyauqi - 18224055

Muhammad Azikra Wira Pratama - 18224089

Fakultas/Sekolah : Sekolah Teknik Elektro dan Informatika

Kode Mata Kuliah : IF3070

Nama Mata Kuliah : Dasar Inteligensi Artifisial

Judul Tugas/Proposal : Tugas Besar 1 IF3070 Dasar Intelegensi Artifisial

Menyatakan bahwa ~~saya~~/kami* menggunakan/~~tidak menggunakan~~* AI dalam pengerjaan maupun penyusunan luaran tugas pada mata kuliah yang tertulis di atas.

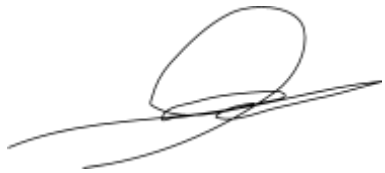
Jika Ya, maka alat AI/kecerdasan buatan yang digunakan adalah:

Lingkup Pekerjaan	Digunakan?	Tingkat Penggunaan
Pemeriksaan Ejaan: menggunakan tools seperti Grammarly, Paperpal, Deepl, Quillbot, Grammarbot, LanguageTool, ProWritingAid, ChatGPT, Google Gemini, atau sejenisnya.	Tidak	
Pembuatan Teks: menggunakan tools seperti ChatGPT, Gemini, Paperpal, Copilot, GrammarlyGO, WordAI, WriteSonic, Jasper, Jenni AI, atau sejenisnya.	Iya	Pembuatan teks sebagai referensi yang kemudian ditulis ulang dalam laporan
Bantuan Diskusi dalam Penyusunan Konten: menggunakan tools seperti ChatGPT, Gemini, Copilot, Perplexity, Jenni AI, Zoom-Companion, atau sejenisnya.	Iya	• Berdiskusi mengenai implementasi algoritma dari salindia • Berdiskusi mengenai syntax Python • Berdiskusi mengenai implementasi

		visualisasi, terutama pada JS dan CSS ● Berdiskusi mengenai cara parsing JSON di Python ● Berdiskusi mengenai debugging, terutama fungsi repair() pada Genetic Algorithm ● Berdiskusi tentang cara setup project
Pembuatan Gambar, Video, dan/atau Grafik: menggunakan tools seperti Craiyon, DALL-E, Midjourney, Stable Diffusion, Microsoft Designer, Gemini, Canva AI, atau sejenisnya.	Tidak	
Lainnya, Jelaskan:		Menggunakan fitur AI Overview di Results Google

~~Saya~~/Kami* juga menyatakan bahwa setiap penggunaan AI/kecerdasan buatan yang dilakukan pada mata kuliah tersebut telah dinyatakan di dalam surat ini.

Bandung, 22 Oktober 2026

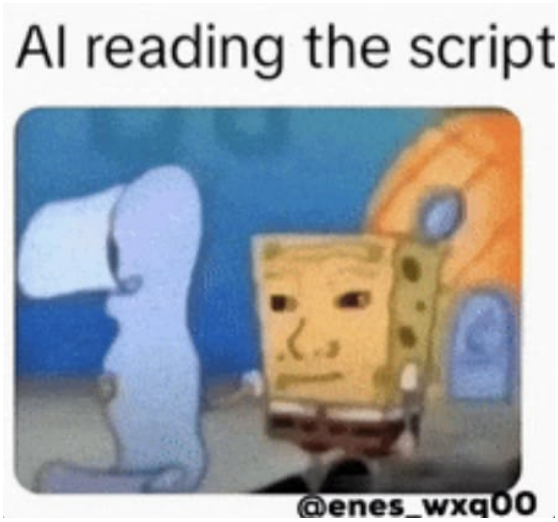
		
Nizham Rafa Lazuardi (18224049)	Adrian Akhdan Assyauqi (18224055)	Muhammad Azikra Wira Pratama (18224089)

Salam dari Kami

Nizham



Adrian



Zikra

